Kylin
NGCC 数字签名第一阶段提交包

Kylin Team

2026 年 4 月 30 日

目录

1 算法描述 **4**

2 相较 SPHINCS$^+$ 的主要设计差异 **5**
2.1 超树中的分层高度多样化 . . . 5
2.2 WOTS$^+$ 编码 . . . 5
2.3 尺寸最优编码 . . . 5
2.4 理论性能 . . . 9

3 记号与支撑性定义 **9**
3.1 数据类型 . . . 9
3.2 函数 . . . 9
3.3 运算 . . . 10
3.4 整数到字节的转换（函数 toByte） . . . 10
3.5 成员函数（函数 set, get） . . . 10
3.6 密码学（哈希）函数族 . . . 11
3.6.1 可调哈希函数（函数 T, F, H） . . . 11
3.6.2 PRF 与消息摘要（函数 PRF, PRF_msg, H_msg） . . . 11
3.6.3 哈希函数地址方案（ADRS 的结构） . . . 11

4 CS-WOTS$^+$ 一次性签名 **13**
4.1 CS-WOTS$^+$ 参数 . . . 13
4.2 CS-WOTS$^+$ 链函数（函数 chain） . . . 13
4.3 CS-WOTS$^+$ 私钥（函数 wots_SKgen） . . . 13
4.4 CS-WOTS$^+$ 公钥生成（函数 wots_PKgen） . . . 14
4.5 CS-WOTS$^+$ 编码（函数 wots_encode） . . . 14
4.6 CS-WOTS$^+$ 签名生成（函数 wots_sign） . . . 14
4.7 CS-WOTS$^+$ 由签名恢复公钥（函数 wots_pkFromSig） . . . 18

5 Kylin 超树 **19**
5.1 （固定输入长度）XMSS . . . 19
5.1.1 XMSS 参数 . . . 19
5.1.2 XMSS 私钥 . . . 20
5.1.3 TreeHash（函数 treehash） . . . 20
5.1.4 XMSS 公钥生成（函数 xmss_PKgen） . . . 20
5.1.5 XMSS 签名 . . . 20
5.1.6 XMSS 签名生成（函数 xmss_sign） . . . 22
5.1.7 由签名恢复 XMSS 公钥（函数 xmss_pkFromSig） . . . 23
5.2 HT：超树 . . . 24
5.2.1 HT 参数 . . . 24
5.2.2 HT 密钥生成（函数 ht_PKgen） . . . 24
5.2.3 HT 签名 . . . 24
5.2.4 HT 签名生成（函数 ht_sign） . . . 24

5.2.5 HT 签名验证（函数 `ht_verify`） 25

6 FORS：随机子集森林 **25**
6.1 FORS 参数 28
6.2 FORS 私钥（函数 `fors_SKgen`） 28
6.3 FORS TreeHash（函数 `fors_treehash`） 28
6.4 FORS 公钥（函数 `fors_PKgen`） 30
6.5 FORS 签名生成（函数 `fors_sign`） 30
6.6 由签名恢复 FORS 公钥（函数 `fors_pkFromSig`） 31

7 完整的 Kylin 签名方案 **31**
7.1 Kylin 参数 31
7.2 Kylin 密钥生成（函数 `kylin_keygen`） 33
7.3 Kylin 签名 34
7.4 Kylin 签名生成（函数 `kylin_sign`） 34
7.5 Kylin 签名验证（函数 `kylin_verify`） 34

8 参数集与哈希/XOF 实例化 **35**
8.1 Kylin 参数集 35
8.1.1 参数对安全性与性能的影响 36
8.1.2 建议参数集与安全级别 37
8.2 哈希函数的实例化 37

9 设计依据 **37**

10 安全性声明与分析 **38**

11 性能评估 **39**

12 特性说明 **40**

A 参数评估 Sage 脚本 **43**

B $q = 2^{24}$ 附加参数配置 **47**

1 算法描述

基于哈希的签名是后量子数字签名中最有前景的候选方向之一，也可能是其中最保守的一条路线。它的优势在于，其经典与量子安全强度主要建立在密码哈希函数理想化困难性之上[1]，因此相较于其他候选方案，其安全性更易理解，也更易评估。

有状态签名。 Ralph Merkle 提出了建立在 Lamport 一次性签名（OTS）之上的基于哈希的签名方案 [21, 19]。随后，对有状态签名的改进工作发展出了扩展 Merkle 签名方案（XMSS） [15] 和 Leighton-Micali 签名（LMS） [20]，并分别被 NIST [8] 与 IETF 标准化。

无状态签名。 在典型的有状态签名中，例如 Merkle 签名方案（MSS），签名者必须记录哪些 OTS 私钥已经使用过，以避免重复使用带来的安全问题。但在许多实际场景中，这样的状态维护并不总是可行。Goldreich 提出了无状态的基于哈希的签名构造 [11, 12]，消除了本地状态的需求，但代价是签名长度极大。近年来，这一研究方向重新受到关注。通过引入超树结构，SPHINCS [4] 给出了 Goldreich 风格无状态哈希签名的一个实用化实例。SPHINCS 进一步成为 NIST PQC 标准化过程中多个后续方案的基础，包括 Gravity-SPHINCS [2]、SPHINCS-Simpira [13] 以及 SPHINCS$^+$ [5]。更近期的优化与分析工作包括 SPHINCS+C [16]、针对超树结构的更优尺寸-时间折中 [17]、可变层高/链长框架扩展 [25]，以及重新审视常和 Winternitz 一次性签名并将其应用于 SPHINCS+ 与 XMSS 的 SPHINCS-alpha 工作 [30]。NIST 还发布了无状态哈希签名的最终联邦标准 FIPS 205（SLH-DSA） [22]。

表 1: 三组已打包 Kylin 家族的哈希调用与尺寸指标。密钥生成、签名和验证以哈希调用代理计数表示；公钥、私钥和签名大小以字节计。

实例	密钥生成	签名	验证	公钥	私钥	签名
Kylin-128	417279	2442880	7357	32	64	6688
Kylin-256	324095	3223168	12577	64	128	26784
Kylin-512	495615	6198272	42476	128	256	103808

我们的工作概述。 我们提出了 Kylin，一种无状态的基于哈希的签名方案。该方案在保留原始 SPHINCS$^+$ 核心结构优势的同时，对这一最先进的无状态哈希签名方案进行了改进。

我们的优化主要来源于底层一次性签名方案的改进。我们证明，这一编码方案在所有树结构一次性签名中都具有尺寸最优性。进一步地，就我们目前所知，没有任何基于 DAG 的构造能够在尺寸效率上优于该方案，因此我们的方案在已知方案中也可视为尺寸最优 [30]。

超树中的分层高度多样化。 Kylin 还采用了更灵活的超树配置：使用分层高度 $(h_0, \ldots, h_{d-1})$，而不是要求所有 d 层具有相同高度。这样可以去除 $h \bmod d = 0$ 这一不必要的整除限制，使参数搜索在总超树高度 h 不能被层数整除时，仍可选择均衡的分层高度向量。

由于我们对 SPHINCS$^+$ 的修改仅限于其底层 WOTS$^+$ 一次性签名的编码方式，以及上述超树分层高度布局，因此新方案的安全分析与原始方案基本一致。基于原始工作的安全分析，我们估算其安全级别，并仔细选择参数以获得更优的整体性能。

[1] 对称原语（包括哈希函数）的设计哲学是：它们应当只允许通用攻击；否则该设计通常会被视为存在缺陷。

为了便于从提交包层面直接查看，表 1 给出了 `Kylin-128`、`Kylin-256` 与 `Kylin-512` 的哈希调用代理指标以及密钥/签名大小。具体而言，该表包含冻结参数集下的密钥生成、签名和验证哈希调用指标。

更完整的参数选择比较见 第11节。如 表 5 所示，已打包的 Kylin 实例给出了当前仓库本地环境下的密钥生成、签名与验证性能概况。对于更重视验证时间的特定场景，还可以进一步将参数重新调整到更有利于快速验证的方向。

致谢。 我们感谢 SPHINCS$^+$ 团队在原始 SPHINCS$^+$ 项目中的出色工作。当前代码与文档中有相当一部分内容均继承自他们值得称道的成果。

2 相较 SPHINCS$^+$ 的主要设计差异

本节重点说明我们相较原始 SPHINCS$^+$ 方案的改进，核心在于其一次性签名组件。我们的优化方案 CS-WOTS$^+$ 相比原始 WOTS$^+$ 方案，能够提供更大的消息空间/签名尺寸比，从而缩小一次性签名的尺寸。由此，整个基于哈希的签名方案可以重新调参，在不显著增加计算时间的前提下进一步减小签名长度。此外，Kylin 在超树上采用了更灵活的分层高度布局，使得在 h 不能被 d 整除时，打包参数仍能保持良好平衡。

本节将解释这些优化的细节，并给出若干技术性证明。

2.1 超树中的分层高度多样化

以往的参数配置通常要求 $h \bmod d = 0$，从而使每一层都具有相同的高度 h/d。在本草稿中，我们允许各层高度 $(h_0, \ldots, h_{d-1})$ 不必完全一致，只要求满足 $\sum_{i=0}^{d-1} h_i = h$。默认情况下，我们假定层高向量按非降序排列，即 $h_0 \le h_1 \le \cdots \le h_{d-1}$。

记 $r = h \bmod d$。我们的默认平衡规则是：

- 取 $d - r$ 层，其高度为 $\lfloor h/d \rfloor$；
- 取 r 层，其高度为 $\lceil h/d \rceil$。

等价地说，该配置尽可能平衡，任意两层高度之差最多为 1。

2.2 WOTS$^+$ 编码

WOTS$^+$（以及其他 Winternitz 类 OTS）之所以引入校验和，是因为如果没有校验和，攻击者只需得到一组合法的消息-签名对，就能够高效伪造新签名。具体来说，给定 (σ, m)，他可以对任意满足 $\forall i(m_i \le m'_i)$ 的消息 m' 进行伪造，因为可以计算 $\mathbf{F}^{m'_i}(sk_i) = \mathbf{F}^{m'_i - m_i}\big(\mathbf{F}^{m_i}(sk_i)\big)$。

校验和正是为了解决这一问题：任一 m_i 的增加都会导致至少一个 C_i 的减小（回忆 $C = \sum_{i=1}^{l_1}(w - 1 - m_i)$）。因此，攻击者不可能同时伪造满足 $\forall i(m_i \le m'_i)$ 且 $\forall i(C_i \le C'_i)$ 的 (m', C')。

2.3 尺寸最优编码

更形式化地说，一次性签名的构造问题可归约为构造一个高效编码函数 $\mathsf{Enc} : \mathcal{M} \to \mathcal{C} \subseteq [w]^l$，其中 $\mathcal{C}$ 是某个互不可比较的码字集合（见 定义 1）。在 WOTS$^+$ 中，编码函数 Enc 只是将校验和附加到原消息之后。注意 WOTS$^+$ 固定了消息大小为 l_1（即 $\mathcal{M} = [w]^{l_1}$），然后在此基础上尽量减小码字长度（最小化 $l - l_1$）。

定义 1 ((不) 可比较性). *对于 $a, b \in [w]^l$，若对每个 $i \in [l]$ 都有 $a_{i+1} \le b_{i+1}$，则记 $a \le b$。若 $a \le b$ 或 $b \le a$，则称 a 与 b 可比较；否则称二者不可比较。若集合 $S \subseteq \{a : a \in [w]^l\}$ 中任意两个元素都不可比较，则称 S 为互不可比较集合（在序理论中也称为“反链”）。*

我们采用了一种稍有不同的消息编码方式。也就是说，我们首先固定码字长度为 l，令 $\mathcal{C} \subseteq [w]^l$，然后尽可能容纳更大的消息空间 $\mathcal{M}$。由于 Enc 是单射，本质上就是要最大化 $\mathcal{C} \subseteq [w]^l$ 的大小。一个自然的思路是让每个码字所有元素之和都等于同一个常数，这样就不再需要显式附加校验和。Bos 和 Chaum [7] 在 $w = 2$ 的特殊情形下研究了这一思路。Vaudenay [28] 将其推广到任意 w，但没有给出编码算法。Perin 等人在 [24] 中给出了类似的编码算法，但没有证明其尺寸最优性。

定理 2.1. *对任意 $m \in [l(w-1)+1]$，$\mathcal{C}_m \stackrel{\text{def}}{=} \{v \in [w]^l : \sum_{i=1}^{l} v_i = m\}$ 都是互不可比较的。*

证明. 反设某个固定 $m \in [l(w-1)+1]$ 下的 $\mathcal{C}_m$ 不是互不可比较的，则存在不同的 $a, b \in \mathcal{C}_m$ 满足 $a \le b$。由于 $a \ne b$，必存在某个指标 j 使得 $a_j < b_j$。然而，由于二者元素和相等，即 $\sum_i a_i = \sum_i b_i$，于是有 $\sum_{1 \le i \le l \wedge i \ne j} (a_i - b_i) > 0$，从而至少存在某个 $1 \le k \le l$ 使得 $a_k > b_k$，这与 $a \le b$ 矛盾。 □ □

每个 $\mathcal{C}_m$ 都对应一种编码方案，但其大小各不相同。当 $m = 0$ 或 $m = l(w-1)$ 时，$\mathcal{C}_m$ 只包含一个码字。我们主张，当 $m = \lfloor \frac{l(w-1)}{2} \rfloor$ 时，$\mathcal{C}_m$ 的大小达到最大。在二进制情形（即 $w = 2$）下，这一点容易验证，因为此时 $|\mathcal{C}_m| = \binom{l}{m}$。Perin 等人 [24] 证明了当 $m = \lfloor \frac{l(w-1)}{2} \rfloor$ 时，$|\mathcal{C}_m|$ 取得最大值。我们在定理 2.2 中给出更强的最优性结果：当 $m = \lfloor \frac{l(w-1)}{2} \rfloor$ 时，$\mathcal{C}_m$ 的大小不仅在所有 $\mathcal{C}_m$（其中 $m \in [l(w-1)+1]$）中最大，而且在所有合法码字集合中也是最大的。

定理 2.2 (尺寸最优编码). *对于任意互不可比较的 $\mathcal{C}^* \in P([w]^l)$，都有*

$$|\mathcal{C}^*| \le |\mathcal{C}_{\lfloor \frac{l(w-1)}{2} \rfloor}| \ .$$

其证明将延后到定理 2.4 中，在那里我们会用序理论语言重新表述定理 2.2。在此之前，我们先讨论如何通过递推计算 $|\mathcal{C}_m|$，并给出一个显式构造，将消息编码到 $m = \lfloor \frac{l(w-1)}{2} \rfloor$ 时的 $\mathcal{C}_m$ 中。此后，为简洁起见，我们将这一最大规模的 $\mathcal{C}_m$ 简记为 $\mathcal{C}$。

计数。 现在我们需要确定 $\mathcal{C}$ 的大小。在特殊情形 $w = 2$ 下，有 $|\mathcal{C}| = \binom{l}{\lfloor l/2 \rfloor}$。固定 w，记

$$D_{n,m} = |\{v \in [w]^n : \sum_{i=1}^{n} v_i = m\}| \ ,$$

则其初值为

$$D_{1,m} = 1, \text{ for } m \in \{0, 1, \ldots, w-1\}$$
$$D_{n,m} = 0, \text{ for } 2 \le n \in \mathbb{Z}, m \in \mathbb{Z}^- \ ,$$

递推关系为

$$D_{n,m} = \sum_{i=0}^{w-1} D_{n-1,m-i}, 2 \le n \in \mathbb{Z}, m \in \{0, 1, \ldots, l(w-1)\} \ . \tag{1}$$

注意当 $w = 2$ 时，这一方法等价于二项式系数的递推关系，即 $\binom{n}{m} = \binom{n-1}{m-1} + \binom{n-1}{m}$。

下面解释这一递推关系。为了计算 $D_{n,m}$，考虑最后一个加数的取值，它可以是 $\{0, 1, \ldots, w-1\}$ 中的任意值。若该值为 i，则前 $n-1$ 个元素之和必须为 $m - i$。因此，“n 个元素求和为 m”的问题可以分解为若干个“$n-1$ 个元素求和为 $m-i$”的子问题。于是只需将所有 $D_{n-1,m-i}$ 累加即可得到 $D_{n,m}$。按照这一方法，$D_{l,\lfloor l(w-1)/2 \rfloor}$ 就给出了 $\mathcal{C}$ 的大小。

相关工作。 $D_{n,m}$ 也是多项式 $(1+x+x^2+\cdots+x^{w-1})^n$ 的第 m 项系数。Euler [10] 研究了 $w=3,4,5$ 的情形, 分别对应三项式、四项式和五项式系数。更一般的形式在文献中也有所研究, 例如 [1, 29, 3]。实际上，我们还可以通过容斥原理将其表示为二项式系数的函数 [14]

$$D_{n,m} = \sum_{s=0}^{\lfloor m/w \rfloor} (-1)^s \binom{n}{s} \binom{m+n-sw-1}{n-1} .$$

编码算法。 下面给出显式构造：一个高效的编码算法，它将消息 $x \in [|\mathcal{C}|]$ 映射到 $\mathcal{C}$ 中的一个元素。该编码算法的伪代码见 算法 1。

算法 1: Encode:$[|\mathcal{C}|] \to \mathcal{C}$.

```
函数 Encode(x)
    令 v 为长度为 l 的数组;
    m ← ⌊l(w − 1)/2⌋;
    for i ← l … 1 do
        for j ← 0 … min(w − 1, m) do
            if x ≥ D_{i−1,s−j} then
                x ← x − D_{i−1,s−j};
            else
                v_{l−i} ← j;
                break;
        m ← m − v_{l−i};
    return v;
```

下面解释这一编码算法。正如前文所述,问题可以通过考虑首个元素 v_{l-i} 的取值划分为多个子问题。为了对自然数 $x \in [0, D_{i,m})$ 进行编码, 我们只需寻找满足 $x \in [\sum_{k<j} D_{i-1,m-k}, \sum_{k\le j} D_{i-1,m-k})$ 的 j，从而确定 $v_{l-i}=j$。一旦 v_{l-i} 的值确定，就继续处理后续各项，直到所有元素都被确定。

为了证明该编码的最优性，我们还需要一些序理论方面的预备知识。一次性签名与序理论之间的关系已经在 [6] 中得到研究。

序理论预备知识

定义 2 (偏序集). *偏序集 $(\mathcal{S}, \le)$ 是一个集合 $\mathcal{S}$ 以及其上的一个二元关系 '$\le$', 该关系满足反对称性、传递性与自反性, 其中某些元素对 $(x,y) \in \mathcal{S}$ 是可比较的（即 $x \le y$ 或 $y \le x$）。*

需要注意的是，偏序集并不要求 $\mathcal{S}$ 中所有元素对都可比较，因此它也被称为“部分有序集”。

定义 3 ((反) 链与分解). *链（chain）是偏序集中任意两元素都可比较的一个子集; 反链（antichain）则是任意两元素都不可比较的一个子集。链分解是将一个偏序集划分为若干互不相交链的分割方式。*

定理 2.3 (Dilworth 定理 [9]). *对任意有限偏序集 P，其最大反链的大小等于其最小链分解的大小。*

令 $P=([w]^l, \le)$ 为一个有限偏序集。根据 Dilworth 定理，我们可以通过以下两点证明 $\mathcal{C}$ 是 P 的最大反链：(1) $\mathcal{C}$ 本身是一个反链；(2) 存在一个链分解，其大小恰好等于 $|\mathcal{C}|$。

定理 2.4. *$\mathcal{C}$ 是最大反链，即它在尺寸上最优。*

证明. 我们已经在定理 2.1 中证明了 $\mathcal{C}$ 是反链。接下来只需构造一个大小为 $|\mathcal{C}|$ 的链分解。我们的证明可以看作 Sperner 定理 [27] 证明的一种推广，后者处理的是 $w=2$ 的特殊情形。

考虑偏序集 $S_n=([w]^n,\leq)$。我们有时将 S_n 中的元素表示为 $(a_1,...,a_n)\in[w]^n$,或记为 $c_i\in[w]^n$。我们略微滥用记号，令 $|(a_1,...,a_n)|\stackrel{\text{def}}{=}a_1+...+a_n$。

我们通过归纳法构造 S_n 的链分解，其中每条链 $\{c_1,\ldots,c_m\}$ 都满足以下两个性质：

- $|c_{i+1}|=|c_i|+1,\forall i\in\{1,2,\ldots,m-1\}$；
- $|c_1|+|c_m|=n\cdot(w-1)$。

当 $n=1$ 时情形显然：$D_{1,\lfloor(w-1)/2\rfloor}=1$，对应于 $S_1=\{(0),(1),\ldots,(w-1)\}$ 这一条链。

假设我们已经构造出满足上述两个性质的 S_{n-1} 链分解，现在来构造 S_n 的链分解。对于 S_{n-1} 链分解中的每条链 $c=\{c_1,c_2,\ldots,c_m\}$, 我们构造 $k+1$ 条属于 S_n 的链, 其中 $k=\min(w-1,m-1)$。也就是说，对每个 $j\in\{0,...,k\}$，第 j 条链由下列元素组成：

$$(c_1,j)\leq\ldots\leq(c_{m-j},j)\leq(c_{m-j},j+1)\leq...\leq(c_{m-j},w-1)\ .$$

从而得到如下图 图 1 所示的 $k+1$ 条链：

$$\begin{array}{ccccccc}(c_1,0)&\ldots&\ldots&\ldots&(c_m,0)&\ldots&(c_m,w-1)\\ \vdots&\ldots&\ldots&\iddots&\ldots&\ldots&\vdots\\ (c_1,k)&\ldots&(c_{m-k},k)&\ldots&\ldots&\ldots&(c_{m-k},w-1)\end{array}$$

图 1: 展示如何将来自 S_{n-1} 的一条链扩展为 S_n 中的 $k+1$ 条链，其中每一行都是一条扩展后的链。注意该图并非矩形矩阵，因为每一行都比上一行少两个元素。

容易验证，$|(c_1,j)|+|(c_{m-j},w-1)|=|(c_1,0)|+j+|(c_m,0)|-j+(w-1)=n(w-1)$，并且链中每个后继元素的和都比前驱元素增加 1。也就是说，这两个性质在 S_n 中构造出的所有链上都得到保持。

接下来还需证明，由 S_{n-1} 各条分解链扩展得到的所有链共同构成 $[w]^n$ 的一个划分。也就是说，对每个 $c_i\in S_{n-1}$，其增广元素 $(c_i,0),...,(c_i,w-1)$ 在构造出的链中恰好各出现一次。注意每个 c_i 必且只能属于 S_{n-1} 链分解中的一条链，例如 $c=\{c_1,\ldots,c_m\}$。我们分情况讨论。

情形 $m\leq w$：此时 $k=m-1\leq w-1$。序列 $[(c_i,0),\ldots,(c_i,k+1-i)]$ 出现在 图 1 的第 i 列前 $(k+2-i)$ 个元素中，而 $[(c_i,k+1-i),\ldots,(c_i,w-1)]^T$ 则出现在第 $(k+2-i)$ 行最后的 $(w+i-k-1)$ 个元素中。

情形 $m>w$：此时 $k=w-1<m-1$。若 $1\leq i\leq m-w+1$，则 $[(c_i,0),(c_i,w-1)]$ 正好构成图 1 的第 i 列。否则, 若 $m-w+1<i\leq m$, 则 $[(c_i,0),\ldots,(c_i,m-i)]$ 与 $[(c_i,m-i),\ldots,(c_i,w-1)]^T$ 分别是第 i 列前 $m-i+1$ 个元素，以及第 $(m-i+1)$ 行最后 $(w+i-m)$ 个元素。

因此，我们已经证明：对于任意 $a\in[w]^{n-1}$，$(a,0)$、...、$(a,w-1)$ 都在新构造出的链中恰好出现一次，即这些链确实构成 S_n 的一个链分解。最后，还需计算该分解中的链数。上述两个性质保证每条链中恰好存在一个元素 c_{mid}，满足 $|c_{\mathsf{mid}}|=\lfloor l(w-1)/2\rfloor$（即 $c_{\mathsf{mid}}\in\mathcal{C}$）。因此，链分解的大小就是 $|\mathcal{C}|=D_{l,\lfloor l(w-1)/2\rfloor}$。这就完成了 $\mathcal{C}$ 为最大反链的证明。

□

表 2: 不同 Winternitz 参数 w 与安全参数 λ 下，WOTS$^+$ 与 CS-WOTS$^+$ 的长度 l 对比。

	128bit		192bit		256bit	
w	WOTS$^+$	Ours	WOTS$^+$	Ours	WOTS$^+$	Ours
8	46	45	67	66	90	88
16	35	34	51	50	67	66
24	31	30	45	44	59	58
32	28	27	42	40	55	53
40	27	26	39	38	52	50
48	25	25	37	36	48	48

2.4 理论性能

常和 WOTS$^+$ 相比 WOTS$^+$ 有两个优点。

- 计算时间稳定。我们的构造中哈希函数调用次数是固定的，而 WOTS$^+$ 的签名与验证算法则可能具有可变调用次数。尽管当前并未发现针对本构造或 WOTS$^+$ 实现的计时攻击，但稳定的计算时间始终更可取，尤其是对于涉及私钥的签名算法。
- 更小的签名尺寸与更少的哈希调用。例如，在一个典型的 256-bit classical 参数选择中，通常会取 $w = 16$ 且 $l = 67$。而在我们的构造中，当 $w = 16$ 时只需 $l = 66$，这会在运行时间（以预期哈希调用次数计）和尺寸上都带来约 1.5% 的降低。更多细节见 表 2。

尽管我们的编码算法相较校验和方法会带来略高的额外开销，但相对于整个签名方案中的哈希函数调用次数而言，这部分开销并不占主导地位；这一点会在后续与 SPHINCS$^+$ 的性能比较中得到印证。

3 记号与支撑性定义

下面我们首先定义整数与比特串上的基本数学操作，再逐步引入后续规范中会用到的更具体的基础方法。

3.1 数据类型

字节与字节串是最基础的数据类型。字节是由八个比特组成的序列，所有字节的集合记为 $\mathbb{B}$。单个字节记为带有前缀 "0x" 的两个十六进制数字。字节串是由零个或多个字节组成的有序序列，记为带前缀 "0x" 的有序十六进制字符序列。例如，0xe534f0 是一个长度为 3 的字节串。字节串数组是一个带下标、从 0 开始的有序集合，其中所有字节串长度相同。本文默认任何数据类型或结构均采用大端表示。

3.2 函数

我们定义以下函数：

$\lceil x \rceil$（或 $\mathsf{ceil}(x)$）在 x 为实数时返回大于等于 x 的最小整数。

$\lfloor x \rfloor$（或 $\mathsf{floor}(x)$）在 x 为实数时返回小于等于 x 的最大整数。

$\log(x)$ 在 x 为非负实数时返回以 2 为底的对数。在伪代码中，这个函数写作 lg。

$\mathsf{Trunc}_\ell(x)$ 将比特串 x 截断为前 ℓ 个比特。

3.3 运算

当 a 与 b 为整数时，数学运算符定义如下：

- ^：a^b 表示 a 的 b 次幂。
- $\cdot$：$a \cdot b$ 表示 a 与 b 的乘积。在无歧义时，该运算符可以按通常数学记号省略。
- /：a/b 表示 a 被非零 b 除的商。
- %：$a\%b$ 表示整数除法的非负余数。
- +：$a+b$ 表示 a 与 b 的和。
- −：$a-b$ 表示 a 与 b 的差。
- ++：$a++$ 表示将 a 增加 1，即 $a=a+1$。
- <<：$a << b$ 表示将 a 逻辑左移 b 位（其中 b 非负），即 $a \cdot 2^b$。
- >>：$a >> b$ 表示将 a 逻辑右移 b 位（其中 b 非负），即 $\mathsf{floor}(a/2^b)$。

在计算算术表达式时，遵循标准运算优先级。

数组按通常方式使用，其中数组 A 的第 i 个元素记为 $A[i]$。在需要时，字节串也被视为字节数组：若 X 是一个字节串，则 $X[i]$ 表示其第 i 个字节，其中 $X[0]$ 为最左侧、最高位字节。

若 A 与 B 是等长字节串，则：

- A AND B 表示按位逻辑与。
- A XOR B（或 $A \oplus B$）表示按位逻辑异或。

当 B 是一个字节、i 是一个整数时，$B >> i$ 表示将其逻辑右移 i 位。

若 X 是一个长度为 x 字节的字节串、Y 是一个长度为 y 字节的字节串，则 $X \| Y$ 表示二者拼接，其中 $X \| Y = X[0]...X[x-1]Y[0]...Y[y-1]$。

3.4 整数到字节的转换（函数 `toByte`）

对于非负整数 x 与 y，定义 $Z = \mathtt{toByte}(x, y)$ 为一个长度为 y 字节的字节串，它以大端字节序存放整数 x 的二进制表示。

3.5 成员函数（函数 `set`，`get`）

为了简化算法描述，我们假定存在成员函数。如果一个复杂数据结构（例如公钥 `PK`）中包含变量 `X`，则 `PK.getX()` 返回该公钥中的 `X` 值。相应地，`PK.setX(Y)` 将 `PK` 中的变量 `X` 设为 `Y` 所持有的值。由于成员函数名采用 camelCase 风格，因此数值 `z` 在函数名中可能写作 `Z`，例如 `getZ`。

3.6 密码学（哈希）函数族

3.6.1 可调哈希函数（函数 T，F，H）

与 SPHINCS$^+$ 一样，一个可调哈希函数除消息输入外，还接受一个公有种子 PK.seed 和一个地址 ADRS 。这样做可使每个密钥对及虚拟树中每个位置上的哈希调用彼此独立。地址方案将在第3.6.3小小节 中说明。

本规范中描述的方案建立在以下形式的若干可调哈希函数实例之上：

$$\mathbf{T}_\ell : \mathbb{B}^n \times \mathbb{B}^{32} \times \mathbb{B}^{\ell n} \to \mathbb{B}^n$$
$$\mathtt{md} \leftarrow \mathbf{T}_\ell(\mathtt{PK.seed}, \mathtt{ADRS}, M) \ ,$$

它使用 n 字节种子 PK.seed 、32 字节地址 ADRS 以及长度为 ℓn 字节的消息 M，输出一个 n 字节哈希值 md。在伪代码中，函数 $\mathbf{T}_\ell$ 记为 T_l。

其中有两个特例，我们沿用以往哈希签名方案中的命名方式：

$$\mathbf{F} : \mathbb{B}^n \times \mathbb{B}^{32} \times \mathbb{B}^n \to \mathbb{B}^n$$
$$\mathbf{F} \stackrel{\text{def}}{=} \mathbf{T}_1$$
$$\mathbf{H} : \mathbb{B}^n \times \mathbb{B}^{32} \times \mathbb{B}^{2n} \to \mathbb{B}^n$$
$$\mathbf{H} \stackrel{\text{def}}{=} \mathbf{T}_2$$

3.6.2 PRF 与消息摘要（函数 PRF，PRF_msg，H_msg）

Kylin 使用伪随机函数 **PRF** 来完成伪随机密钥生成：

$$\mathbf{PRF} : \mathbb{B}^n \times \mathbb{B}^{32} \to \mathbb{B}^n \ .$$

此外，Kylin 使用伪随机函数 $\mathbf{PRF}_{\mathbf{msg}}$ 为消息压缩生成随机性：

$$\mathbf{PRF}_{\mathbf{msg}} : \mathbb{B}^n \times \mathbb{B}^n \times \mathbb{B}^* \to \mathbb{B}^n \ .$$

为了压缩待签消息，Kylin 还使用一个能够处理任意长度消息的带密钥哈希函数 $\mathbf{H}_{\mathbf{msg}}$：

$$\mathbf{H}_{\mathbf{msg}} : \mathbb{B}^n \times \mathbb{B}^n \times \mathbb{B}^n \times \mathbb{B}^* \to \mathbb{B}^m \ .$$

3.6.3 哈希函数地址方案（ADRS 的结构）

地址 ADRS 是一个 32 字节值，具有规定的结构，并配套提供若干 set 方法来操作地址。后续各节在使用这些地址时会说明具体生成方式。其本质是：所有函数都必须跟踪当前上下文，并在每次哈希调用之后更新地址。

针对不同用途，一共定义了七种不同类型的地址。其中一种用于 CS-WOTS$^+$ 中的哈希，一种用于 CS-WOTS$^+$ 公钥压缩，第三种用于主 Merkle 树中的哈希，第四种用于 FORS 中 Merkle 树内的哈希，第五种用于 FORS 树根压缩，最后两种分别用于 CS-WOTS$^+$ 与 FORS 的私钥值生成。这些地址大多共享相同的总体格式，下面将详细说明。

地址结构遵循字边界；在这里，一个字宽为 32 位。只有 tree address（即主树中特定子树的索引）长度过长，无法放入单个字，因此我们为它保留三个字。总体而言，地址结构如下：最高位开始是一个字的 layer address，后接三个字的 tree address。这两个字段描述了一棵树在超树中的位

置。layer address 描述该树在超树中的层高位置，自底层树开始记为第 0 层；tree address 描述该层中具体是哪一棵树，从最左边的树开始编号为 0。其后一个字表示地址类型：CS-WOTS$^+$ 哈希地址取 0，CS-WOTS$^+$ 公钥压缩取 1，哈希树地址取 2，FORS 地址取 3，FORS 树根压缩取 4。

我们先介绍 CS-WOTS$^+$ 地址（图 2）。在这种情况下，类型字之后是 key pair address，用来编码指定树内 CS-WOTS$^+$ 密钥对的索引。再下一个字编码 chain address（即 CS-WOTS$^+$ 内链的索引），最后一个字编码该链内部哈希函数调用的位置。需要注意的是，基于 `SK.seed` 生成私钥时使用的是另一种地址类型（见后文）。

Layer Address	Tree Address		
type = 0	Key Pair Address	Chain Address	Hash Address

图 2: CS-WOTS$^+$ 哈希地址。

第二种类型（图 3）用于压缩 CS-WOTS$^+$ 公钥。其类型字取 1。与 CS-WOTS$^+$ 内部使用的地址类似，下一字编码 key pair address。剩余两个字不需要，因此保持为 0。我们用 0 将地址补齐到固定的 32 字节长度。

Layer Address	Tree Address	
type = 1	Key Pair Address	Padding = 0

图 3: CS-WOTS$^+$ 公钥压缩地址。

第三种类型（图 4）用于主树中的哈希函数调用。此时类型字取 2，其后是一个字的 0 填充。再后一个字编码当前正在计算的树节点高度，最后一个字编码该高度处节点的索引。

Layer Address	Tree Address		
type = 2	Padding = 0	Tree Height	Tree Index

图 4: 哈希树地址。

下一种类型（图 5）格式相似，用于描述 FORS 树中的哈希函数。其类型字取 3。key pair address 用于标识使用的是哪一个 FORS 密钥对，其含义与 CS-WOTS$^+$ 哈希地址中的 key pair address 相同；它的值等于用于认证该 FORS 密钥对的 CS-WOTS$^+$ 密钥对的索引，即其在指定树中的叶子编号。tree height 与 tree index 字段则用于定位 FORS 树内的哈希调用，方式与主树中的哈希类似，只是这里的 tree index 在不同 FORS 树之间连续编号。基于 `SK.seed` 生成叶节点时，使用另一种地址类型（见后文）。

再下一种类型（图 6）用于压缩 FORS 树根。其类型字取 4。与 CS-WOTS$^+$ 公钥压缩地址类似，它只包含 FORS 密钥对地址，其余部分补零至完整长度。

最后两种类型分别用于 CS-WOTS$^+$ 和 FORS 的私钥值生成。CS-WOTS$^+$ 私钥生成地址（图 7）与 CS-WOTS$^+$ 哈希地址相同，仅有两点不同：其一，类型字取 5；其二，hash address 字恒设为 0。当为某条链生成私钥值时，其余各字段必须与该链对应的 CS-WOTS$^+$ 哈希地址保持一致。

类似地，FORS 私钥生成类型（图 8）与 FORS 树地址类型相同，只是类型字取 6，且 tree height 字恒设为 0。和 CS-WOTS$^+$ 私钥生成地址一样，其余字段必须与处理该值时所用 FORS 树地址保持一致。

Layer Address	Tree Address		
type = 3	Key Pair Address	Tree Height	Tree Index

图 5: FORS 树地址。

Layer Address	Tree Address	
type = 4	Key Pair Address	Padding = 0

图 6: FORS 树根压缩地址。

地址中的所有字段都编码无符号整数。描述地址生成时，我们使用 set 方法：它接受正整数，并将某一字段的比特设置为该整数的大端二进制表示。本文统一采用如下约定：当改变地址的类型字（即调用 `setType()` 方法）时，后续三个字会被初始化为 0。

为了便于在后续伪代码中跟踪这些地址类型，我们分别用常量 `WOTS_HASH`、`WOTS_PK`、`TREE`、`FORS_TREE`、`FORS_ROOTS`、`WOTS_PRF` 和 `FORS_PRF` 来引用它们。

4 CS-WOTS$^+$ 一次性签名

本节描述 CS-WOTS$^+$ 方案。虽然一个私钥理论上能够对任意消息进行签名，但每个私钥 **绝不能**用于签名多于一条消息。

4.1 CS-WOTS$^+$ 参数

CS-WOTS$^+$ 使用参数 n, w 和 `len`；它们均为正整数。参数含义如下：

- n：安全参数，同时也是消息长度以及私钥、公钥、签名元素的字节长度；
- w：Winternitz 参数；
- `len`：CS-WOTS$^+$ 私钥、公钥与签名中 n 字节元素的个数。

4.2 CS-WOTS$^+$ 链函数（函数 `chain`）

链函数（算法 2）在一个 n 字节输入上迭代计算 **F**，并使用 CS-WOTS$^+$ 哈希地址 `ADRS` 和公有种子 `PK.seed`。地址 `ADRS` 的前七个 32 位字 **必须**编码该链的地址。在每次迭代中，`ADRS` 在用于调用 **F** 处理输入前，都会先被更新为当前链内位置。

下面，`ADRS` 是 第3.6.3小小节 中定义的一个 32 字节 CS-WOTS$^+$ 哈希地址，`PK.seed` 是一个 n 字节字符串。链函数的输入包括：一个 n 字节字符串 X、起始索引 i、迭代步数 s，以及 `ADRS` 和 `PK.seed`。输出则是在输入 X 上迭代 s 次 **F** 后得到的值。

4.3 CS-WOTS$^+$ 私钥（函数 `wots_SKgen`）

CS-WOTS$^+$ 私钥记为 `SK` , 是一个长度为 `len` 的 n 字节串数组。该私钥 **绝不能**用于签名多于一条消息。这个私钥仅在逻辑上被使用，因此下面的内容主要是为了帮助理解后续算法。CS-WOTS$^+$ 私钥中的每个 n 字节串都由秘密种子 `SK.seed` 和一个 CS-WOTS$^+$ 私钥生成地址 `skADRS` 通过 **PRF**

Layer Address	Tree Address		
type = 5	Key Pair Address	Chain Address	Hash Address = 0

图 7: CS-WOTS$^+$ 私钥生成地址。

Layer Address	Tree Address		
type = 6	Key Pair Address	Tree Height = 0	Tree Index

图 8: FORS 私钥生成地址。

推导而来；其中 SK.seed 是 Kylin 私钥的一部分。同一个秘密种子会用于生成 Kylin 内部所有私钥值。用于生成 SK 中第 i 个 n 字节串的地址，**必须**编码该 CS-WOTS$^+$ 实例在 Kylin 结构中的第 i 条哈希链位置。

下面的伪代码（算法 3）描述了 CS-WOTS$^+$ 私钥的生成过程。

4.4 CS-WOTS$^+$ 公钥生成（函数 wots_PKgen）

CS-WOTS$^+$ 密钥对定义了一个虚拟结构，它由 len 条长度为 w 的哈希链组成。私钥中每个 n 字节串都对应一条哈希链的起始节点。公钥则是这些哈希链终止节点的可调哈希压缩值。计算这些哈希链时使用链函数（算法 2）。调用方需要提供 CS-WOTS$^+$ 哈希地址 ADRS 、公有种子 PK.seed 以及秘密种子 SK.seed 。地址 ADRS **必须**编码该 CS-WOTS$^+$ 密钥对在 Kylin 结构中的位置。因此，CS-WOTS$^+$ 算法 **不得**操作 ADRS 除最后三个 32 位字以外的任何部分。需要注意的是，这里使用的 PK.seed 是公开信息，验证者同样可以获得。

下面的伪代码（算法 4）描述了公钥 PK 的生成过程。

4.5 CS-WOTS$^+$ 编码（函数 wots_encode）

下面解释编码算法（算法 5）。该问题可以通过考察首个元素 c_{l-i} 的取值被划分为若干子问题。为了对一个自然数 $x \in [0, D_{i,m})$ 编码，我们只需寻找满足 $x \in [\sum_{k<j} D_{i-1,m-k}, \sum_{k\leq j} D_{i-1,m-k})$ 的 j，并据此确定 $c_{l-i} = j$。当 v_{l-i} 的值确定后，就继续处理后续元素，直到所有元素都确定。所有算术运算 **必须**在无符号 256 位整数上完成。数组 D 可以预计算，从而避免每次调用编码函数时都重复计算。

4.6 CS-WOTS$^+$ 签名生成（函数 wots_sign）

CS-WOTS$^+$ 签名是一个长度为 len 的 n 字节串数组。生成过程首先将消息 M 映射为 len 个介于 0 与 $w-1$ 之间的整数，并要求这些整数之和等于 $\lfloor \texttt{len} \cdot (w-1)/2 \rfloor$。调用方需要提供 CS-WOTS$^+$ 哈希地址 ADRS 、公有种子 PK.seed 和秘密种子 SK.seed 。该地址会编码此 CS-WOTS$^+$ 密钥对在更大结构中的位置，因此 CS-WOTS$^+$ 算法 **不得**修改 ADRS 除最后三个 32 位字以外的任何部分。需要注意的是，这里使用的 PK.seed 对验证者也是公开信息，而秘密种子 SK.seed 则是私有信息。生成 CS-WOTS$^+$ 签名 SIG 的伪代码见下（算法 6）。

签名数据格式见 图 9。

算法 2: chain — CS-WOTS$^+$ 中使用的链函数。

```
# Input: Input string X, start index i, number of steps s, public seed
↪ PK.seed, address ADRS
# Output: value of F iterated s times on X
chain(X, i, s, PK.seed, ADRS) {
  if ( s == 0 ) {
    return X;
  }
  if ( (i + s) > (w - 1) ) {
    return NULL;
  }
  byte[n] tmp = chain(X, i, s - 1, PK.seed, ADRS);
  ADRS.setHashAddress(i + s - 1);
  tmp = F(PK.seed, ADRS, tmp);
  return tmp;
}
```

算法 3: wots_SKgen — 生成 CS-WOTS$^+$ 私钥。

```
# Input: secret seed SK.seed, address ADRS
# Output: CS-WOTS+ private key sk
wots_SKgen(SK.seed, ADRS) {
  skADRS = ADRS; // copy address to create key generation address
  skADRS.setType(WOTS_PRF);
  skADRS.setKeyPairAddress(ADRS.getKeyPairAddress());
  for ( i = 0; i < len; i++ ) {
    skADRS.setChainAddress(i);
    skADRS.setHashAddress(0);
    sk[i] = PRF(SK.seed, skADRS);
  }
  return sk;
}
```

算法 4: wots_PKgen — 生成 CS-WOTS$^+$ 公钥。

```
# Input: secret seed SK.seed, address ADRS, public seed PK.seed
# Output: CS-WOTS+ public key pk
wots_PKgen(SK.seed, PK.seed, ADRS) {
  wotspkADRS = ADRS; // copy address to create OTS public key address
  skADRS = ADRS; // copy address to create key generation address
  skADRS.setType(WOTS_PRF);
  skADRS.setKeyPairAddress(ADRS.getKeyPairAddress());
  for ( i = 0; i < len; i++ ) {
  skADRS.setChainAddress(i);
    skADRS.setHashAddress(0);
    sk[i] = PRF(SK.seed, skADRS);
    ADRS.setChainAddress(i);
    ADRS.setHashAddress(0);
    tmp[i] = chain(sk[i], 0, w - 1, PK.seed, ADRS);
  }
  wotspkADRS.setType(WOTS_PK);
  wotspkADRS.setKeyPairAddress(ADRS.getKeyPairAddress());
  pk = T_len(PK.seed, wotspkADRS, tmp);
  return pk;
}
```

算法 5: wots_encode — 将消息编码为常和表示。

```
#Input: Message M
#Output: Constant-Sum Encoding of M
wots_encode(M) {

  s=floor(len*(w-1)/2)
  Let D be an 2d array of size (len+1)*(s+1) initialized to 0

  D[0][0]=1
  for( i = 1 ; i <= len ; i++ ){
    for( j = 0 j <= s ; j++ ){
      for( k = 0 ; k < w and k <= j ; k++ ){
        D[i][j] = D[i][j] + D[i-1][j-k]
      }
    }
  }
  // array D can be precomputed

  Let c be an array of size len
  for( i = len ; i >= 1 ; i-- ){
    for( j = 0 ; j < w and j <= s  ; j++ ){
      if( M >= D[i-1][s-j] ){
        M = M - D[i-1][s-j]
      } else {
        c[len-i] = j
        break
      }
    }
    s = s - c[len-i]
  }

  return c;
}
```

算法 6: wots_sign — 对消息 M 生成 CS-WOTS$^+$ 签名。

```
# Input: Message M, secret seed SK.seed, public seed PK.seed, address ADRS
# Output: CS-WOTS+ signature sig
wots_sign(M, SK.seed, PK.seed, ADRS) {
  msg = wots_encode(M);
  skADRS = ADRS; // copy address to create key generation address
  skADRS.setType(WOTS_PRF);
  skADRS.setKeyPairAddress(ADRS.getKeyPairAddress());
  for ( i = 0; i < len; i++ ) {
    skADRS.setChainAddress(i);
    skADRS.setHashAddress(0);
    sk = PRF(SK.seed, skADRS);
    ADRS.setChainAddress(i);
    ADRS.setHashAddress(0);
    sig[i] = chain(sk, 0, msg[i], PK.seed, ADRS);
  }
  return sig;
}
```

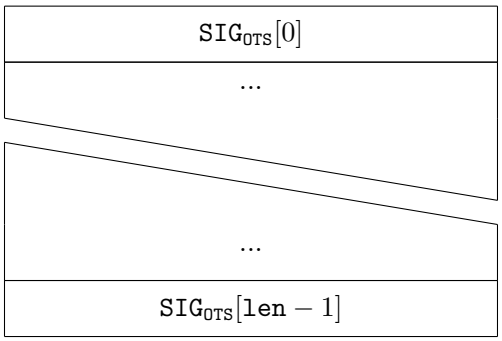


图 9: CS-WOTS$^+$ 签名数据格式。

4.7 CS-WOTS$^+$ 由签名恢复公钥（函数 wots_pkFromSig）

Kylin 对 CS-WOTS$^+$ 使用隐式签名验证。为了验证消息 M 上的 CS-WOTS$^+$ 签名 SIG，验证者需要从签名恢复一个 CS-WOTS$^+$ 公钥值。这可以通过从签名值继续“补全”链计算来完成，其中使用的步数由消息哈希及其校验和的 base-w 表示决定。这个步骤称为 wots_pkFromSig，其过程见算法 7。随后还需要对 wots_pkFromSig 的结果进行验证。若该方案单独使用，则可直接通过比较完成；而在 Kylin 中，则通过利用该结果继续计算 Kylin 公钥来实现隐式验证。

调用方需要提供 CS-WOTS$^+$ 哈希地址 ADRS 和公有种子 PK.seed。地址将编码该 CS-WOTS$^+$ 密钥对在 Kylin 结构中的位置。因此，CS-WOTS$^+$ 算法 **不得**操作 ADRS 除最后三个 32 位字之外的部分。这里使用的 PK.seed 对验证者同样是公开信息。

算法 7: `wots_pkFromSig` — 由消息及其签名恢复 CS-WOTS$^+$ 公钥。

```
# Input: Message M, CS-WOTS+ signature sig, address ADRS, public seed
↪  PK.seed
# Output: CS-WOTS+ public key pk_sig derived from sig
wots_pkFromSig(sig, M, PK.seed, ADRS) {
  wotspkADRS = ADRS;
  msg = wots_encode(M)
  for ( i = 0; i < len; i++ ) {
    ADRS.setChainAddress(i);
    tmp[i] = chain(sig[i], msg[i], w - 1 - msg[i], PK.seed, ADRS);
  }
  wotspkADRS.setType(WOTS_PK);
  wotspkADRS.setKeyPairAddress(ADRS.getKeyPairAddress());
  pk_sig = T_len(PK.seed, wotspkADRS, tmp);
  return pk_sig;
}
```

5 Kylin 超树

本节解释 Kylin 超树的构造方式。我们首先说明如何将 CS-WOTS$^+$ 与二叉哈希树结合，从而得到一个固定输入长度版本的 XMSS。随后再说明如何在此基础上扩展为超树。超树可以被视作固定输入长度版的多树 XMSS（XMSS^{MT} ）。

5.1 （固定输入长度）XMSS

XMSS 用于对一个可能很大但固定数量的消息集合进行签名。它建立在 Merkle 签名方案之上，通过一棵高度为 h_{XMSS} 的二叉树来认证 $2^{h_{\mathrm{XMSS}}}$ 个 CS-WOTS$^+$ 公钥。因此，高度为 h_{XMSS} 的 XMSS 密钥对可用于签名 $2^{h_{\mathrm{XMSS}}}$ 条不同消息。二叉树中的每个节点都是一个 n 字节值，由其两个子节点拼接后的可调哈希得到。叶子节点即 CS-WOTS$^+$ 公钥。XMSS 公钥就是整棵树的根节点。在 Kylin 中，XMSS 私钥表现为单一秘密种子，它被用于生成所有 CS-WOTS$^+$ 私钥。

在 Kylin 上下文中，XMSS 签名由两部分组成：消息上的 CS-WOTS$^+$ 签名，以及所谓的认证路径。后者是一个树节点向量，使验证者能够从 CS-WOTS$^+$ 签名出发计算出树根值。验证者据此恢复根节点并验证其正确性。一个独立的 XMSS 签名通常还包含所使用 CS-WOTS$^+$ 密钥对的索引；但在 Kylin 中，这不是必须的，因为整个 Kylin 签名已经足以恢复其中每个 XMSS 签名对应的索引。

5.1.1 XMSS 参数

XMSS 具有以下参数：

- h_{XMSS}：树的高度（即“层数减 1”）。
- n：消息以及每个节点的字节长度。

- w：Winternitz 参数，其定义与前一节中的 CS-WOTS$^+$ 相同。

树中共有 $2^{h_{\text{XMSS}}}$ 个叶子。XMSS 签名记为 $\texttt{SIG}_{\texttt{XMSS}}$ （伪代码中为 `SIG_XMSS`）。CS-WOTS$^+$ 签名记为 `SIG` 。

XMSS 参数在需要时隐式包含于算法输入中。

5.1.2 XMSS 私钥

在 Kylin 的上下文中，XMSS 私钥就是 Kylin 私钥中包含的单一秘密种子 `SK.seed` 。它用于按第4节 所述方式生成 XMSS 密钥对结构中的 CS-WOTS$^+$ 私钥。

5.1.3 TreeHash（函数 `treehash`）

为了计算 Merkle 树内部的 n 字节节点，子程序 `treehash`（算法 8）接受以下输入：秘密种子 `SK.seed` 、公有种子 `PK.seed`、无符号整数 s（起始索引）、无符号整数 z（目标节点高度），以及编码包含该子树地址的 `ADRS` 。树中节点高度从叶子层开始记为 0。`treehash` 算法返回一棵高度为 z 的子树根节点，其中最左侧叶子对应索引 s 上的 CS-WOTS$^+$ 公钥。这里 **要求** $s \,\%\, 2^z = 0$，也就是说，索引为 s 的叶子必须是某棵高度为 z 的子树的最左叶；否则算法会失败，因为它将试图计算并不存在的节点。这里描述的 `treehash` 算法使用一个至多保存 $(z-1)$ 个节点的栈，并假定存在标准栈操作 `push()` 与 `pop()`。此外，我们默认节点的高度（无符号整数）与节点值（n 字节串）一同存储在栈中。

5.1.4 XMSS 公钥生成（函数 `xmss_PKgen`）

XMSS 公钥按 `xmss_PKgen`（算法 9）中的方式计算。在 Kylin 中，XMSS 公钥 PK 即为二叉哈希树的根节点。根节点通过 `treehash` 计算得到。公钥生成算法输入为秘密种子 `SK.seed` 、公有种子 `PK.seed` 以及地址 `ADRS` ，后者编码该 XMSS 实例在 Kylin 结构中的位置。

5.1.5 XMSS 签名

XMSS 签名是一个长度为 $((\texttt{len} + h_{\text{XMSS}}) \cdot n)$ 字节的字符串，由以下两部分组成：

- 一个 CS-WOTS$^+$ 签名 $\texttt{SIG}_{\texttt{OTS}}$ ，占用 $\texttt{len} \cdot n$ 字节；
- 与所用 CS-WOTS$^+$ 密钥对对应叶子相关联的认证路径 `AUTH`，占用 $h_{\text{XMSS}} \cdot n$ 字节。

认证路径是一个由 h_{XMSS} 个 n 字节串构成的数组。它包含了从所用叶子到根路径上各节点的兄弟节点，但不包含路径上的节点本身。验证者正是利用这些 `AUTH` 中的节点，从 CS-WOTS$^+$ 公钥出发恢复树根节点。一个节点 N 由其在树中的位置来表示：$N(x, y)$ 表示第 x 层上的第 y 个节点，其中 $y = 0$ 表示该层最左节点。叶子位于第 0 层，根位于第 h_{XMSS} 层。认证路径在每个层 $0 \le x \le (h_{\text{XMSS}} - 1)$ 上恰好包含一个节点。对于从 0 开始计数的第 i 个 CS-WOTS$^+$ 密钥对，其第 j 个认证路径节点为

$$\texttt{AUTH}[j] = N\left(j, \lfloor \frac{i}{2^j} \rfloor \oplus 1\right) \ .$$

认证路径的计算方法见 第5.1.6小小节。签名数据格式见 图 10。

算法 8: `treehash` — TreeHash 算法。

```
# Input: Secret seed SK.seed, start index s, target node height z, public
↪  seed PK.seed, address ADRS
# Output: n-byte root node - top node on Stack
treehash(SK.seed, s, z, PK.seed, ADRS) {
  if( s % (1 << z) != 0 ) return -1;
  for ( i = 0; i < 2^z; i++ ) {
    ADRS.setType(WOTS_HASH);
    ADRS.setKeyPairAddress(s + i);
    node = wots_PKgen(SK.seed, PK.seed, ADRS);
    ADRS.setType(TREE);
    ADRS.setTreeHeight(1);
    ADRS.setTreeIndex(s + i);
    while ( Top node on Stack has same height as node ) {
      ADRS.setTreeIndex((ADRS.getTreeIndex() - 1) / 2);
      node = H(PK.seed, ADRS, (Stack.pop()  node));
      ADRS.setTreeHeight(ADRS.getTreeHeight() + 1);
    }
    Stack.push(node);
  }
  return Stack.pop();
}
```

算法 9: `xmss_PKgen` — 生成 XMSS 公钥。

```
# Input: Secret seed SK.seed, public seed PK.seed, address ADRS
# Output: XMSS public key PK
xmss_PKgen(SK.seed, PK.seed, ADRS) {
  pk = treehash(SK.seed, 0, h_XMSS, PK.seed, ADRS)
  return pk;
}
```

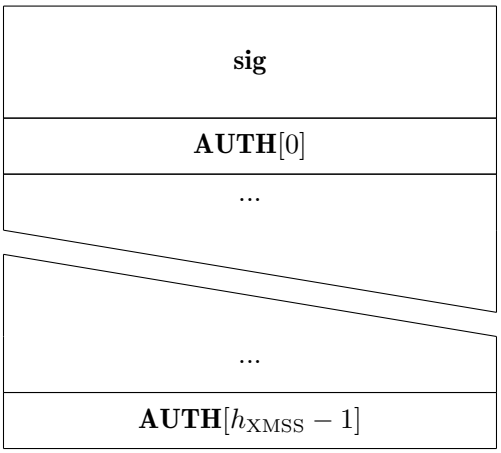


图 10: XMSS 签名。

5.1.6 XMSS 签名生成（函数 xmss_sign）

在 Kylin 的上下文中，为消息 M 计算 XMSS 签名需要以下输入：秘密种子 SK.seed 、公有种子 PK.seed、待使用的 CS-WOTS$^+$ 密钥对索引 idx，以及 XMSS 实例地址 ADRS 。首先，使用索引为 idx 的 CS-WOTS$^+$ 实例对消息摘要进行签名；随后再计算认证路径。

认证路径中的节点值 *可以*通过任意方式计算。最节省内存的方法是使用 treehash 算法（算法 8）逐个计算所有节点，这也是这里的描述方式。需要注意的是，这一步具体如何实现与互操作性无关；为了执行签名验证，实际上不必了解这些实现细节。

算法 10: xmss_sign — 生成 XMSS 签名。

```
# Input: n-byte message M, secret seed SK.seed, index idx, public seed
  ↪ PK.seed, address ADRS
# Output: XMSS signature SIG_XMSS = (sig || AUTH)
xmss_sign(M, SK.seed, idx, PK.seed, ADRS) {
  // build authentication path
  for ( j = 0; j < h_XMSS; j++ ) {
    k = floor(idx / (2^j)) XOR 1;
    AUTH[j] = treehash(SK.seed, k * 2^j, j, PK.seed, ADRS);
  }
  ADRS.setType(WOTS_HASH);
  ADRS.setKeyPairAddress(idx);
  sig = wots_sign(M, SK.seed, PK.seed, ADRS);
  SIG_XMSS = sig  AUTH;
  return SIG_XMSS;
}
```

5.1.7 由签名恢复 XMSS 公钥（函数 xmss_pkFromSig）

Kylin 使用 XMSS 签名的隐式验证方式。一个 XMSS 签名会被用来计算一个候选 XMSS 公钥，也就是树的根节点。这个根值会继续参与后续计算（例如更上一层树对它的签名），并在该后续计算的结果中被隐式验证。因此，本规范不包含独立的 xmss_verify 方法，而是提供 xmss_pkFromSig 方法。

xmss_pkFromSig 的输入包括：一个 n 字节消息 M、一个 XMSS 签名 $\mathtt{SIG}_{\mathtt{XMSS}}$、签名索引 idx、公有种子 PK.seed 以及地址 ADRS。其中 ADRS 编码了当前 XMSS 实例在 Kylin 密钥对虚拟结构中的位置。算法首先调用 wots_pkFromSig 恢复候选 CS-WOTS$^+$ 公钥，再结合认证路径计算出根节点，并将其返回。xmss_pkFromSig 算法见 算法 11。

算法 11: xmss_pkFromSig — 由 XMSS 签名恢复 XMSS 公钥。

```
# Input: index idx, XMSS signature SIG_XMSS = (sig || AUTH), n-byte message
↪ M, public seed PK.seed, address ADRS
# Output: n-byte root value node[0]
xmss_pkFromSig(idx, SIG_XMSS, M, PK.seed, ADRS){ // compute WOTS+ pk from
↪ WOTS+ sig
ADRS.setType(WOTS_HASH);
ADRS.setKeyPairAddress(idx);
sig = SIG_XMSS.getWOTSSig();
AUTH = SIG_XMSS.getXMSSAUTH();
node[0] = wots_pkFromSig(sig, M, PK.seed, ADRS);
// compute root from WOTS+ pk and AUTH
ADRS.setType(TREE);
ADRS.setTreeIndex(idx);
for ( k = 0; k < h_XMSS; k++ ) {
  ADRS.setTreeHeight(k+1);
  if ( (floor(idx / (2^k)) % 2) == 0 ) {
      ADRS.setTreeIndex(ADRS.getTreeIndex() / 2);
      node[1] = H(PK.seed, ADRS, (node[0]  AUTH[k]));
    } else {
      ADRS.setTreeIndex((ADRS.getTreeIndex() - 1) / 2);
      node[1] = H(PK.seed, ADRS, (AUTH[k]  node[0]));
    }
    node[0] = node[1];
  }
  return node[0];
}
```

5.2 HT：超树

Kylin 的超树 HT 是 XMSS^{MT} 的一个变体。本质上，它是一棵由 XMSS 实例构成的认证树。HT 由多层 XMSS 树组成：顶层和中间层的树用来签名其下一层 XMSS 树的公钥（即根节点），而最底层的树则用于签名实际消息；在 Kylin 中，这里的“实际消息”具体是 FORS 公钥。

考虑一棵总高度为 h 的 HT，其包含 d 层 XMSS 树，自底向上的层高分别为 $(h_0, \ldots, h_{d-1})$，满足 $\sum_{i=0}^{d-1} h_i = h$。那么第 $d-1$ 层包含一棵 XMSS 树。一般地，第 i 层包含 $2^{\sum_{j=i+1}^{d-1} h_j}$ 棵 XMSS 树。最后，第 0 层包含 2^{h-h_0} 棵 XMSS 树。

5.2.1 HT 参数

除 XMSS 的全部参数外，HT 还需要超树总高度 h、树层数 d，以及满足 $\sum_{i=0}^{d-1} h_i = h$ 的各层 XMSS 高度 $(h_0, \ldots, h_{d-1})$。所有层都使用相同的 Winternitz 参数 w。

5.2.2 HT 密钥生成（函数 `ht_PKgen`）

HT 私钥就是秘密种子 `SK.seed`，它被用来生成 HT 虚拟结构中所有 CS-WOTS^{+} 私钥。

HT 公钥则是顶层唯一那棵 XMSS 树的公钥（根节点）。其计算过程如下。公钥生成算法以私有种子和公有种子为输入。

算法 12: `ht_PKgen` — 生成 HT 公钥。

```
# Input: Private seed SK.seed, public seed PK.seed
# Output: HT public key PK_HT
ht_PKgen(SK.seed, PK.seed){
      ADRS = toByte(0, 32);
      ADRS.setLayerAddress(d-1);
      ADRS.setTreeAddress(0);
      root = xmss_PKgen(SK.seed, PK.seed, ADRS);
      return root;
}
```

5.2.3 HT 签名

HT 签名 $\mathtt{SIG}_{\mathtt{HT}}$ 是一个长度为 $(h + d \cdot \mathtt{len}) \cdot n$ 的字节串。它由 d 个 XMSS 签名组成，其中第 i 层贡献一个长度为 $(h_i + \mathtt{len}) \cdot n$ 字节的 XMSS 签名。

签名数据格式见 图 11。

5.2.4 HT 签名生成（函数 `ht_sign`）

为了对消息 M 计算 HT 签名 $\mathtt{SIG}_{\mathtt{HT}}$，下面的 `ht_sign`（算法 13）会使用 第5.1.6小小节 中定义的 `xmss_sign`。该算法输入为消息 M、秘密种子 `SK.seed`、公有种子 `PK.seed` 以及索引 `idx`。这个索引标识超树中将用于对消息签名的叶子。最终的 HT 签名由一叠 XMSS 签名组成，覆盖从索引为 idx 的叶子一路到顶层树路径上的 XMSS 树。需要注意，idx 实际上传递时被拆分为两个独立参数：一个用于定位具体树的 tree index，另一个用于该树内部的 leaf index。这样可以支持更高的

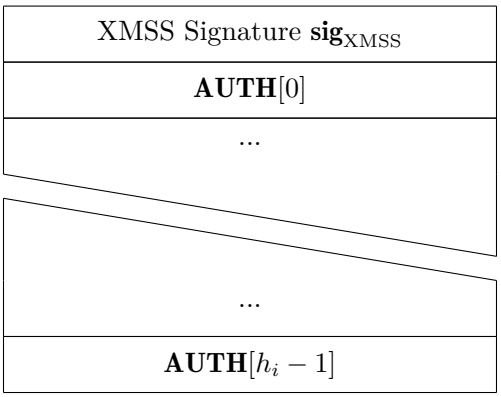


图 11: HT 签名。

超树，因为参数允许在 $h < 64 + h_0$ 的范围内使用 64 位整数表示 `tree_idx`。这与本文规范中的参数一致。若使用允许更大 h 的其他参数集，则 `tree_idx` 的数据类型 **必须**相应调整。

算法 `ht_sign` 会在某个 XMSS 实例完成签名后，使用 `xmss_pkFromSig` 来恢复该实例的根节点。另一种方式是使用 `xmss_PKgen`。不过，`xmss_PKgen` 会重建整棵树，而 `xmss_pkFromSig` 在第 i 层只需一次 `wots_pkFromSig` 调用和 $(h_i - 1)$ 次 **H** 调用。因此，下面给出的 `ht_sign` 只是生成 HT 签名的一种方式；只要输出相同，*也可以*使用其他方法。

5.2.5 HT 签名验证（函数 `ht_verify`）

HT 签名验证（算法 14）可以概括为：调用 d 次 `xmss_pkFromSig`，再与给定值进行一次比较。HT 签名验证的输入包括：消息 M、签名 $\texttt{SIG}_{\texttt{HT}}$、公有种子 `PK.seed`、索引 `idx`（同样拆分为 tree index 与 leaf index），以及 HT 公钥 $\texttt{PK}_{\texttt{HT}}$。

6 FORS：随机子集森林

Kylin 的超树 HT 并不直接签名原始消息，而是签名 FORS 实例的公钥；而 FORS 公钥又用于签名消息摘要。FORS（forest of random subsets，随机子集森林）是一种少次签名方案（FTS）。FORS 改进自 HORST [4]，而 HORST 本身又是 HORS [26] 的一个变体。从安全性角度看，FORS 的输入必须是哈希函数的输出。下文将 FORS 描述为作用在比特串上的签名方案。

FORS 使用参数 k 和 $t = 2^a$（例如可取 $t = 2^{15}, k = 10$）。FORS 对长度为 ka 比特的字符串进行签名。这里我们不再统一使用字节作为长度单位，因为消息长度的比特数未必是 8 的倍数。私钥由 kt 个随机的 n 字节串构成，并分成 k 组，每组包含 t 个 n 字节串。私钥值通过 Kylin 私钥中的主秘密种子 `SK.seed` 伪随机生成。在 Kylin 中，FORS 私钥值仅作为计算公钥或签名时的中间结果被临时生成。

FORS 公钥是一个单独的 n 字节哈希值。它通过对 k 棵二叉哈希树的根节点做可调哈希压缩得到。这 k 棵二叉哈希树的高度均为 a，每棵树用于认证一组中的 t 个私钥值。因此，树的叶子就是其私钥组中各值的（可调）哈希。

一个对字符串 M 的签名由 k 个私钥值及其对应的认证路径组成，每组私钥元素恰取一个。为生成签名，首先将 `md` 划分为 k 个长度为 a 比特的字符串。由于 `md` 是字节序列，我们先按字节枚举，再在每个字节内部按从低位到高位的顺序将其展开为比特串。随后，将这些比特串分别解释为介于 0 与 $t - 1$ 之间的整数。每个整数都用于在对应私钥组中选择一个私钥值。也就是说，若第一

算法 13: `ht_sign` — 生成 HT 签名。

```
# Input: Message M, private seed SK.seed, public seed PK.seed, tree index
↪  idx_tree, leaf index idx_leaf
# Output: HT signature SIG_HT
ht_sign(M, SK.seed, PK.seed, idx_tree, idx_leaf) {
  // init
  // per-layer XMSS heights are h_0, h_1, ..., h_{d-1} with sum h
  ADRS = toByte(0, 32);
  // sign
  ADRS.setLayerAddress(0);
  ADRS.setTreeAddress(idx_tree);
  SIG_tmp = xmss_sign(M, SK.seed, idx_leaf, PK.seed, ADRS);
  SIG_HT = SIG_HT  SIG_tmp;
  root = xmss_pkFromSig(idx_leaf, SIG_tmp, M, PK.seed, ADRS);
  for ( j = 1; j < d; j++ ) {
    idx_leaf = h_j least significant bits of idx_tree;
    idx_tree = idx_tree >> h_j;
    ADRS.setLayerAddress(j);
    ADRS.setTreeAddress(idx_tree);
    SIG_tmp = xmss_sign(root, SK.seed, idx_leaf, PK.seed, ADRS);
    SIG_HT = SIG_HT  SIG_tmp;
    if ( j < d - 1 ) {
      root = xmss_pkFromSig(idx_leaf, SIG_tmp, root, PK.seed, ADRS);
    }
  }
  return SIG_HT;
}
```

算法 14: `ht_verify` — 使用 HT 公钥 $\mathbf{PK}_{HT}$ 验证消息 M 上的 HT 签名 $\mathbf{SIG}_{\mathbf{HT}}$。

```
# Input: Message M, signature SIG_HT, public seed PK.seed, tree index
↪  idx_tree, leaf index idx_leaf, HT public key PK_HT.
# Output: Boolean
ht_verify(M, SIG_HT, PK.seed, idx_tree, idx_leaf, PK_HT){
  // init
  // per-layer XMSS heights are h_0, h_1, ..., h_{d-1} with sum h
  ADRS = toByte(0, 32);
  // verify
  SIG_tmp = SIG_HT.getXMSSSignature(0);
  ADRS.setLayerAddress(0);
  ADRS.setTreeAddress(idx_tree);
  node = xmss_pkFromSig(idx_leaf, SIG_tmp, M, PK.seed, ADRS);
  for ( j = 1; j < d; j++ ) {
    idx_leaf = h_j least significant bits of idx_tree;
    idx_tree = idx_tree >> h_j;
    SIG_tmp = SIG_HT.getXMSSSignature(j);
    ADRS.setLayerAddress(j);
    ADRS.setTreeAddress(idx_tree);
    node = xmss_pkFromSig(idx_leaf, SIG_tmp, node, PK.seed, ADRS);
    }
  if ( node == PK_HT ) {
    return true;
  } else {
    return false;
  }
}
```

个整数是 i，则选取第一组中的第 i 个私钥元素，依此类推。最终签名由这些被选中的私钥元素及其认证路径组成。

Kylin 对 FORS 使用隐式验证：它只提供一个根据签名恢复候选公钥的方法。恢复过程是：利用输入字符串计算得到的索引、签名中的私钥值以及认证路径，计算出 k 棵树的根节点；再对这些根节点做可调哈希，得到候选公钥并返回。

下面开始描述 FORS 的参数与算法。

6.1 FORS 参数

FORS 使用参数 n, k 和 t；它们均为正整数。参数含义如下：

- n：安全参数，即私钥、公钥或签名元素的字节长度。
- k：私钥集合的个数、树的个数，以及从输入字符串中计算出的索引个数。
- t：每个私钥集合中的元素个数、每棵哈希树的叶子个数，以及索引值的上界。参数 t **必须**是 2 的幂。若 $t = 2^a$，则对应树高为 a，输入字符串会被切分成长度为 a 的比特串。

FORS 的输入是长度为 $k \log t$ 的比特串。

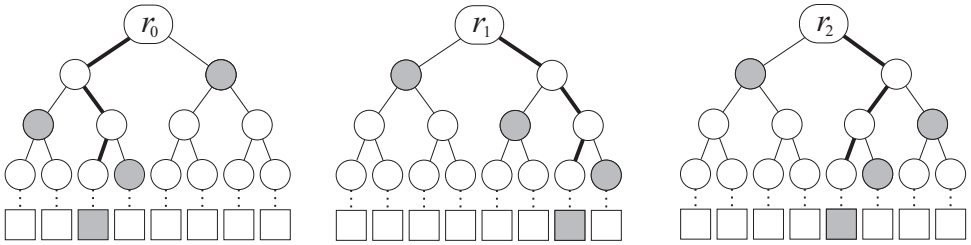


图 12: FORS 树与公钥。

6.2 FORS 私钥（函数 `fors_SKgen`）

在 Kylin 的上下文中，FORS 私钥就是 Kylin 私钥中的单一秘密种子 `SK.seed` 。利用 FORS 私钥生成地址和 **PRF** ，可以由该种子生成 kt 个 n 字节私钥值。虽然从逻辑上看这些值应当分组为一个二维数组，但在实现时通常可以将其视为一个长度为 kt 的一维数组。此时，第 i 组中的第 j 个元素存放在 $\texttt{SK}[it + j]$。为了生成其中某个元素，需要使用一个 FORS 私钥生成地址 `skADRS`，该地址编码了 FORS 密钥对在 Kylin 中的位置，并将 tree height 设为 0，leaf index 设为 $it + j$：

6.3 FORS TreeHash（函数 `fors_treehash`）

在讨论 FORS 公钥之前，我们先说明树的计算方式。为了计算 FORS 哈希树中的 n 字节节点，使用子程序 `fors_treehash`。它本质上与 第5.1小节 中的 `treehash`（算法 8）是同一个算法，区别只在于叶节点如何计算以及地址如何处理。不过由于地址结构非常相似，实现时完全可以较容易地将两者合并为一个通用例程。

算法 `fors_treehash` 接受以下输入：秘密种子 `SK.seed` 、公有种子 `PK.seed`、无符号整数 s（起始索引）、无符号整数 z（目标节点高度），以及编码 FORS 密钥对地址的 `ADRS` 。与 `treehash` 类似，`fors_treehash` 返回一棵高度为 z 的树的根节点，其中最左叶子对应索引 s 的私钥元素经哈希后的值。这里的 s 是在整个 kt 个私钥元素上连续编号的。并且 **要求** $s \,\%\, 2^z = 0$，即索引为 s 的叶子必须是某棵高度为 z 子树的最左叶；否则算法会失败，因为它将尝试计算并不存在的节点。

算法 15: fors_SKgen — 计算 FORS 私钥值。

```
#Input: secret seed SK.seed, address ADRS, secret key index idx = it+j
#Output: FORS private key sk
fors_SKgen(SK.seed, ADRS, idx) {
  skADRS = ADRS; // copy address to create key generation address
  skADRS.setType(FORS_PRF);
  skADRS.setKeyPairAddress(ADRS.getKeyPairAddress());

  skADRS.setTreeHeight(0);
  skADRS.setTreeIndex(idx);
  sk = PRF(SK.seed, skADRS);
  return sk;
}
```

算法 16: fors_treehash 算法。

```
# Input: Secret seed SK.seed, start index s, target node height z, public
↪  seed PK.seed, address ADRS
# Output: n-byte root node - top node on Stack
fors_treehash(SK.seed, s, z, PK.seed, ADRS) {
  if( s % (1 << z) != 0 ) return -1;
  for ( i = 0; i < 2^z; i++ ) {
    sk = fors_SKgen(SK.seed, ADRS, s+i)
    node = F(PK.seed, ADRS, sk);
    ADRS.setTreeHeight(1);
    ADRS.setTreeIndex(s + i);
    while ( Top node on Stack has same height as node ) {
      ADRS.setTreeIndex((ADRS.getTreeIndex() - 1) / 2);
      node = H(PK.seed, ADRS, (Stack.pop()  node));
      ADRS.setTreeHeight(ADRS.getTreeHeight() + 1);
    }
    Stack.push(node);
  }
  return Stack.pop();
}
```

6.4 FORS 公钥（函数 fors_PKgen）

在 Kylin 的上下文中，FORS 公钥从不会被单独生成；它只会与签名一起出现。我们在此保留 fors_PKgen，主要是为了完整性、便于理解以及测试。算法 fors_PKgen 输入秘密种子 SK.seed 、公有种子 PK.seed 和一个 FORS 地址 ADRS ；后者编码该 FORS 实例在 Kylin 中的位置。算法输出一个 FORS 公钥。

算法 17: fors_PKgen — 生成 FORS 公钥。

```
# Input: Secret seed SK.seed, public seed PK.seed, address ADRS
# Output: FORS public key PK
fors_PKgen(SK.seed, PK.seed, ADRS) {
  forspkADRS = ADRS; // copy address to create FTS public key address
  for(i = 0; i < k; i++){
    root[i] = fors_treehash(SK.seed, i*t, a, PK.seed, ADRS);
  }
  forspkADRS.setType(FORS_ROOTS);
  forspkADRS.setKeyPairAddress(ADRS.getKeyPairAddress());
  pk = T_k(PK.seed, forspkADRS, root);
  return pk;
}
```

6.5 FORS 签名生成（函数 fors_sign）

FORS 签名是一个长度为 $k(\log t + 1)$ 的 n 字节串数组。它包含 k 个私钥值（每个 n 字节），以及与之对应的认证路径（每条路径包含 $\log t$ 个 n 字节值）。

算法 fors_sign 的输入是一个长度为 $(k \log t)$ 比特的字符串 M、秘密种子 SK.seed 、公有种子 PK.seed 和地址 ADRS 。该地址编码了 FORS 实例在 Kylin 中的位置。算法输出一个 FORS 签名 $\mathtt{SIG}_{\mathtt{FORS}}$。

签名数据格式见 图 13。

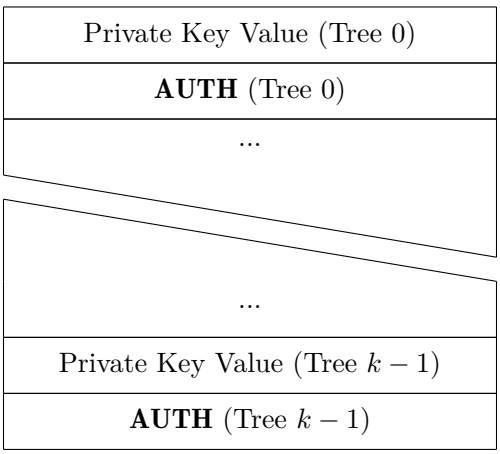


图 13: FORS 签名。

算法 18: fors_sign — 对字符串 M 生成 FORS 签名。

```
# Input: Bit string M, secret seed SK.seed, address ADRS, public seed
↪ PK.seed
# Output: FORS signature SIG_FORS
fors_sign(M, SK.seed, PK.seed, ADRS) {
  // compute signature elements
  for(i = 0; i < k; i++){
    // get next index
    unsigned int idx = bits i*log(t) to (i+1)*log(t) - 1 of M;
    // pick private key element
    SIG_FORS = SIG_FORS  fors_SKgen(SK.seed, ADRS, i*t + idx) ;
    // compute auth path
    for ( j = 0; j < a; j++ ) {
      s = floor(idx / (2^j)) XOR 1;
      AUTH[j] = fors_treehash(SK.seed, i * t + s * 2^j, j, PK.seed, ADRS);
    }
    SIG_FORS = SIG_FORS  AUTH;
  }
  return SIG_FORS;
}
```

6.6 由签名恢复 FORS 公钥（函数 fors_pkFromSig）

Kylin 对 FORS 签名采用隐式验证方式。一个 FORS 签名会被用来恢复一个候选 FORS 公钥。该公钥将继续参与后续计算（作为上层 XMSS 树签名的消息），并在后续计算结果中被隐式验证。因此，本规范不包含 fors_verify 方法，而是提供 fors_pkFromSig 方法。

fors_pkFromSig 的输入包括：一个长度为 $k \log t$ 比特的字符串 M、一个 FORS 签名 $\mathtt{SIG}_{\mathtt{FORS}}$、公有种子 PK.seed 以及地址 ADRS 。该地址编码了 FORS 实例在 Kylin 密钥对虚拟结构中的位置。算法首先利用 fors_treehash 计算出 k 棵二叉哈希树的根节点，随后再用可调哈希函数 T_k 对这些根节点做压缩。fors_pkFromSig 算法见 算法 19。该方法会调用 $\mathtt{SIG}_{\mathtt{FORS}}$.getSK(i) 与 $\mathtt{SIG}_{\mathtt{FORS}}$.getAUTH(i)：前者返回签名中存储的第 i 个私钥元素，后者返回签名中存储的第 i 条认证路径。

7 完整的 Kylin 签名方案

到这里，我们已经具备描述主构造 Kylin 所需的全部组件。本质上，Kylin 沿用了 Kylin 的基础结构，唯一的差异在于其一次性签名组件。

7.1 Kylin 参数

Kylin 具有以下参数：

- n：以字节计的安全参数。

算法 19: `fors_pkFromSig` — 由 FORS 签名恢复 FORS 公钥。

```
# Input: FORS signature SIG_FORS, (k lg t)-bit string M, public seed
↪ PK.seed, address ADRS
# Output: FORS public key
fors_pkFromSig(SIG_FORS, M, PK.seed, ADRS){
  // compute roots
  for(i = 0; i < k; i++){
    // get next index
    unsigned int idx = bits i*log(t) to (i+1)*log(t) - 1 of M;
    // compute leaf
    sk = SIG_FORS.getSK(i);
    ADRS.setTreeHeight(0);
    ADRS.setTreeIndex(i*t + idx);
    node[0] = F(PK.seed, ADRS, sk);
    // compute root from leaf and AUTH
    auth = SIG_FORS.getAUTH(i);
    ADRS.setTreeIndex(i*t + idx);
    for ( j = 0; j < a; j++ ) {
      ADRS.setTreeHeight(j+1);
      if ( (floor(idx / (2^j)) % 2) == 0 ) {
        ADRS.setTreeIndex(ADRS.getTreeIndex() / 2);
        node[1] = H(PK.seed, ADRS, (node[0]  auth[j]));
      } else {
        ADRS.setTreeIndex((ADRS.getTreeIndex() - 1) / 2);
        node[1] = H(PK.seed, ADRS, (auth[j]  node[0]));
      }
      node[0] = node[1];
    }
    root[i] = node[0];
  }
  forspkADRS = ADRS; // copy address to create FTS public key address
  forspkADRS.setType(FORS_ROOTS);
  forspkADRS.setKeyPairAddress(ADRS.getKeyPairAddress());
  pk = T_k(PK.seed, forspkADRS, root);
  return pk;
}
```

- w：第4.1小节 中定义的 Winternitz 参数。
- h：第5.2.1小小节 中定义的超树高度。
- d：第5.2.1小小节 中定义的超树层数。
- k：第6.1小节 中定义的 FORS 树数量。
- t：第6.1小节 中定义的 FORS 树叶子数量。

前文给出的全部限制在这里都同样适用。回忆我们记 $a = \log t$。此外，由这些参数还可导出 m 和 `len`：

- m：消息摘要长度（单位：字节），其计算公式为

$$m = \lfloor (k \log t + 7)/8 \rfloor + \lfloor (h - h_0 + 7)/8 \rfloor + \lfloor (h_0 + 7)/8 \rfloor.$$

  虽然理论上只需要 $h + k \log t$ 个比特，但采用这里定义的更长 m 能显著简化实现。

- `len`：CS-WOTS$^+$ 私钥、公钥与签名中 n 字节字符串元素的个数。选择 `len` 时，需要保证下列偏序集的大小大于目标消息空间 $\{0,1\}^n$（参见 第2节）：

$$\left| D_{\texttt{len}, \lfloor \frac{\texttt{len} \cdot (w-1)}{2} \rfloor} \right| \geq 2^n \ .$$

下文假设所有算法都可访问这些参数。

7.2 Kylin 密钥生成（函数 kylin_keygen）

Kylin 私钥包含两个元素。第一，n 字节秘密种子 `SK.seed`，用于生成所有 CS-WOTS$^+$ 与 FORS 私钥元素。第二，n 字节 PRF 密钥 `SK.prf`，用于确定性地产生随机化消息哈希所需的随机值。

Kylin 公钥同样包含两个元素。第一，HT 公钥，即顶层树的根节点。第二，一个均匀随机采样得到的 n 字节公有种子值 `PK.seed`。

由于 `kylin_sign` 并不直接接收公钥，但又需要访问 `PK.seed`（以及在缓解故障攻击时可能还需要 `PK.root`），因此 Kylin 私钥内部还包含了一份公钥副本。

算法 `kylin_keygen` 的描述假定存在一个函数 `sec_rand`，其输入为整数 i，输出为 i 字节的高强度密码学随机数。

算法 20: `kylin_keygen` — 生成 Kylin 密钥对。

```
# Input: (none)
# Output: Kylin key pair (SK,PK)
kylin_keygen( ){
  SK.seed = sec_rand(n);
  SK.prf = sec_rand(n);
  PK.seed = sec_rand(n);
  PK.root = ht_PKgen(SK.seed, PK.seed);
  return ( (SK.seed, SK.prf, PK.seed, PK.root), (PK.seed, PK.root) );
}
```

Kylin 私钥与公钥的格式见 图 14。

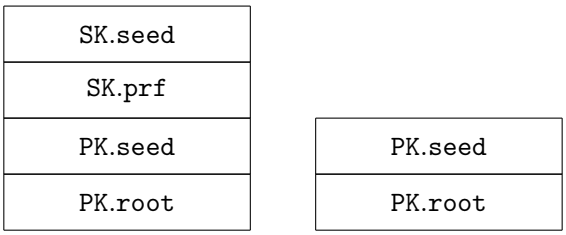


图 14: 左：Kylin 私钥；右：Kylin 公钥。

7.3 Kylin 签名

Kylin 签名 $\texttt{SIG}_{\texttt{HT}}$ 是一个长度为 $(1 + k \cdot (a + 1) + h + d \cdot \texttt{len})n$ 的字节串。它由一个 n 字节随机化字符串 R 、一个由 $k(a + 1)n$ 字节组成的 FORS 签名 $\texttt{SIG}_{\texttt{FORS}}$，以及一个长度为 $(h + d \cdot \texttt{len})n$ 字节的 HT 签名 $\texttt{SIG}_{\texttt{HT}}$ 组成。

签名数据格式见 图 15。

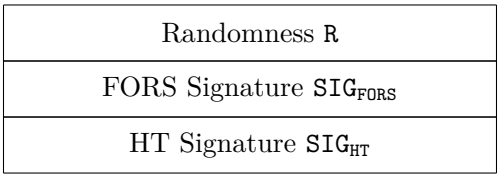


图 15: Kylin 签名。

7.4 Kylin 签名生成（函数 kylin_sign）

生成 Kylin 签名分为四步。第一步，伪随机生成随机值 R。第二步，利用该随机值计算一个长度为 m 字节的消息摘要，并将其拆分为三部分：长度为 $\lfloor (k \log t + 7)/8 \rfloor$ 字节的部分消息摘要 tmp_md、长度为 $\lfloor (h - h_0 + 7)/8 \rfloor$ 字节的树索引 tmp_idx_tree，以及长度为 $\lfloor (h_0 + 7)/8 \rfloor$ 字节的叶索引 tmp_idx_leaf。第三步, 通过抽取所需比特数, 进一步得到实际值 md、idx_tree 和 idx_leaf。随后，使用最低 HT 层中第 idx_tree 棵 XMSS 树上的第 idx_leaf 个 FORS 密钥对对部分消息摘要 md 进行签名。最后，再使用 HT 对该 FORS 公钥进行签名。需要注意的是，完整索引并不会被整体使用，而是在实现中会立刻拆分成 tree index 与 leaf index。

在计算 R 时，PRF 接受一个 n 字节字符串 opt。它默认初始化为 PK.seed，但若全局变量 RANDOMIZE 被设置，也可以改为使用外部随机性。保留这一选项的原因在于，否则 Kylin 签名将始终是确定性的。

7.5 Kylin 签名验证（函数 kylin_verify）

Kylin 的签名验证（算法 22）可以概括为：重新计算消息摘要与索引、恢复一个候选 FORS 公钥，并验证该公钥上的 HT 签名。注意，若该候选 FORS 公钥与真实公钥不一致，则 HT 签名验证将在压倒性概率下失败。Kylin 签名验证的输入包括：消息 M、签名 SIG 以及 Kylin 公钥 PK。

8 参数集与哈希/XOF 实例化

本节讨论 Kylin 的实例化方式。Kylin 可被视为一种签名模板：通过对所使用的密码学函数族进行实例化，就可得到具体的签名系统。我们将密码函数族的不同实现方式视为不同签名系统。与函数族实例化正交的是参数集问题：参数集为 Kylin 的各项参数赋予具体数值。

本节首先给出参数所需满足的条件，并讨论不同参数在安全性、尺寸与速度之间所带来的权衡；随后再汇总面向 NGCC 的冻结参数集，以及已打包实现所使用的哈希/XOF 实例化。

8.1 Kylin 参数集

Kylin 由前文已经描述过的以下参数刻画；所有参数均为正整数。

- n：以字节计的安全参数。
- w：Winternitz 参数。
- h：超树高度。
- d：超树层数。
- k：FORS 中树的数量。
- t：FORS 树的叶子数。

回忆我们记 $a = \log t$。此外，由这些参数可以导出 m 和 `len`：

- m：消息摘要长度（字节），其计算公式为

$$m = \lfloor (k \log t + 7)/8 \rfloor + \lfloor (h - h_0 + 7)/8 \rfloor + \lfloor (h_0 + 7)/8 \rfloor.$$

- `len`：CS-WOTS$^+$ 私钥、公钥与签名中 n 字节字符串元素的个数。

$$\texttt{len} = \min \left\{ \ell : \left| D_{\ell, \lfloor \frac{\ell \cdot (w-1)}{2} \rfloor} \right| \geq 2^n \right\} \ ,$$

其中 $D_{\ell, \lfloor \frac{\ell \cdot (w-1)}{2} \rfloor}$ 的大小通过 式 (1) 中的递推关系来计算。

下面我们重新总结这些参数的作用、要求及性质，并给出若干公式，精确说明它们对性能与安全性的影响。安全参数 n 也是除 $\mathbf{H}_{\mathbf{msg}}$ 之外所有密码函数族的输出长度。因此，它在很大程度上决定一个参数集能够达到的安全级别。同时，它也是 Kylin 结构中几乎所有节点的大小，因此所有签名元素的长度都是 n 的整数倍。

Winternitz 参数 w 决定了每个 CS-WOTS$^+$ 实例中哈希链的条数与长度。增大 w 会线性增加哈希链长度，但对链条数量的减少只带来对数级影响。哈希链条数正好对应于 CS-WOTS$^+$ 签名中 n 字节值的个数，因此它对 Kylin 签名尺寸有显著影响。哈希链数量与长度的乘积又与签名速度直接相关，因为 HT 签名生成的主要时间几乎都花在计算 CS-WOTS$^+$ 公钥上。因此，较大的 w 意味着更短的签名，但更慢的签名过程。不过要注意其中的指数差异：w 越大，进一步缩短签名所付出的代价就越高。Winternitz 参数本身不影响 Kylin 的安全性。

超树高度 h 决定了 FORS 实例的数量，因此也决定了在给定 Kylin 密钥对签名次数下，某个 FORS 密钥对被重复使用多次的概率。所以，高度 h 直接影响安全性：超树越高，安全性越强；但另一方面，树越高，签名也越大。

层数 d 是一个纯粹的性能折中参数，并不直接影响安全性。它决定了超树中 XMSS 树的层数。各层高度 $(h_0, \ldots, h_{d-1})$ 则决定每层 XMSS 的高度，并必须满足 $\sum_{i=0}^{d-1} h_i = h$。增大 d 往往有利于缩小子树规模并加快签名；但 d 同时也控制层数，从而控制 HT 中 CS-WOTS$^+$ 签名的数量，进而影响整个 Kylin 签名长度。

参数 k 与 t 决定 FORS 的性能与安全性。FORS 树的叶子数 t 必须是 2 的幂，而 k 可以自由选择。通常较小的 t 会带来更小、更快的签名；但在给定安全级别下，较小的 t 往往需要较大的 k，这又会增大签名尺寸并拖慢签名速度。因此，这两个参数之间需要平衡，而最佳平衡通常应依赖下文给出的公式。

消息摘要长度 m 是 $\mathbf{H}_{\mathbf{msg}}$ 的输出字节长度,其值为 $\lfloor (k \log t + 7)/8 \rfloor + \lfloor (h - h_0 + 7)/8 \rfloor + \lfloor (h_0 + 7)/8 \rfloor$ 字节。

CS-WOTS$^+$ 密钥对中链条数量 `len` 决定了 CS-WOTS$^+$ 签名的尺寸。

8.1.1 参数对安全性与性能的影响

下面我们给出用于计算给定 Kylin 参数集在速度、尺寸和安全性方面表现的公式，以支持参数选择。附录 A 中还提供了一个 SAGE 脚本。

密钥生成。 生成 Kylin 私钥和 `PK.seed` 需要调用三次安全随机数生成器。随后需要生成顶层树（第 $d-1$ 层）。叶节点部分需要进行 $2^{h_{d-1}}$ 次 CS-WOTS$^+$ 密钥生成（生成 `SK` 需要 `len` 次 PRF 调用，生成 `PK` 需要 w`len` 次 **F** 调用），并对 CS-WOTS$^+$ 公钥进行压缩（一次 $\mathbf{T}_{\texttt{len}}$ 调用）。计算顶层树根则还需要 $(2^{h_{d-1}} - 1)$ 次 **H** 调用。

签名。 随机化和消息压缩分别需要一次 $\mathbf{PRF}_{\mathbf{msg}}$ 和一次 $\mathbf{H}_{\mathbf{msg}}$ 调用。FORS 签名需要 kt 次 **PRF** 和 **F** 调用；此外还需计算 k 棵高度为 $\log t$ 的二叉树的根，这会增加 $k(t-1)$ 次 **H** 调用，最后还需要一次 $\mathbf{T}_k$ 调用。接着，需要计算一次 HT 签名，而 HT 签名由 d 层树组成，其结构与密钥生成中的树类似。因此，总共需要 $\sum_{i=0}^{d-1} 2^{h_i}$ 次 `len` 次 PRF 调用与 w`len` 次 **F** 调用，以及 $\sum_{i=0}^{d-1} 2^{h_i}$ 次 $\mathbf{T}_{\texttt{len}}$ 调用。计算每棵树的根还会额外带来 $\sum_{i=0}^{d-1}(2^{h_i} - 1)$ 次 **H** 调用。

验证。 首先需要用 $\mathbf{H}_{\mathbf{msg}}$ 计算消息哈希。随后执行一次 FORS 验证：其中需要 k 次 **F** 调用（由签名元素恢复叶节点），$k \log t$ 次 **H** 调用（结合叶节点和认证路径恢复根节点），以及一次 $\mathbf{T}_k$ 调用来压缩这些根。接着，需要验证 d 个 XMSS 签名；每个签名在 CS-WOTS$^+$ 验证阶段需要少于 w`len` 次 **F** 调用以及一次 $\mathbf{T}_{\texttt{len}}$ 调用[2]。除此之外，d 个根恢复总共还需要 $\sum_{i=0}^{d-1} h_i = h$ 次 **H** 调用。

Kylin 的私钥、公钥与签名大小分别为 $4n$、$2n$ 和 $(h + k(a+1) + d \cdot \texttt{len} + 1)n$。

Kylin 在面对通用攻击时的经典安全级别（bit security）可由下式计算：

$$\texttt{bitsec} = -\log\left(\frac{1}{2^{8n}} + \sum_{\gamma}\left(1 - \left(1 - \frac{1}{t}\right)^{\gamma}\right)^{k}\binom{q}{\gamma}\left(1 - \frac{1}{2^h}\right)^{q-\gamma}\frac{1}{2^{h\gamma}}\right) .$$

Kylin 在面对通用攻击时的量子安全级别可由下式计算：

$$\texttt{bitsec} = -\frac{1}{2}\log\left(\frac{1}{2^{8n}} + \sum_{\gamma}\left(1 - \left(1 - \frac{1}{t}\right)^{\gamma}\right)^{k}\binom{q}{\gamma}\left(1 - \frac{1}{2^h}\right)^{q-\gamma}\frac{1}{2^{h\gamma}}\right) .$$

这里同样忽略了对数内部的小常数因子。

[2]需要说明的是，w`len` 次 **F** 调用是一个最坏情况上界，它对应 CS-WOTS$^+$ 签名验证的最坏开销。由于待签消息本身是哈希值，可视为接近均匀分布，因此在实际测量中，这一数值通常更接近平均情况上界 $(w/2) \cdot$ `len`。

8.1.2 建议参数集与安全级别

正如上一小节所述，即便在固定安全级别下，Kylin 的设计也支持在签名尺寸与速度之间作出不同折中。对于本提交包，我们采用 表 3 中列出的三组 NGCC 对齐 Kylin 实例。这些实例由仓库中的参数脚本生成，并针对征集要求中的 128/256/512 bit 经典安全级别以及相应的量子安全目标进行了调优。

表 3: 用于已打包主实现的冻结 NGCC 对齐 Kylin 参数族。

参数族	NGCC 等级	q	n	h/d	$(h_0,\ldots,h_{d-1})$	a	k	w	`len`	bitsec	pk	sk	sig
Kylin-128	128/80	2^{64}	16	65/9	(7, 7, 7, 7, 7, 7, 7, 8, 8)	13	11	73	22	127	32	64	6688
Kylin-256	256/128	2^{64}	32	68/10	(6, 6, 7, 7, 7, 7, 7, 7, 7, 7)	13	22	54	46	256	64	128	26784
Kylin-512	512/256	2^{64}	64	66/11	(6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6)	13	49	97	79	511	128	256	103808

8.2 哈希函数的实例化

在 NGCC 提交包中，哈希与 XOF 调用遵循打包实现所使用的 ICCS 辅助接口。具体而言，当前包使用 `sm3hash`、`pseudohash` 与 `pseudoXOF`，并在 `hash_iccs.c` 与 `thash_iccs.c` 模块中基于 `pseudoXOF` 构建带域分离的包装函数。

回忆 n 和 m 分别表示安全参数以及消息摘要长度（单位：字节）。

在实现层面，其映射关系如下：

$$
\begin{aligned}
\mathbf{H}_{\mathbf{msg}}(\mathtt{R},\mathtt{PK.seed},\mathtt{PK.root},M) &= \mathrm{pseudoXOF}(\mathtt{tag}_{H_msg}\|\mathtt{R}\|\mathtt{PK.seed}\|\mathtt{PK.root}\|M,m),\\
\mathbf{PRF}(\mathtt{PK.seed},\mathtt{SK.seed},\mathtt{ADRS}) &= \mathrm{pseudoXOF}(\mathtt{tag}_{PRF}\|\mathtt{PK.seed}\|\mathtt{ADRS}\|\mathtt{SK.seed},n),\\
\mathbf{PRF}_{\mathbf{msg}}(\mathtt{SK.prf},\mathtt{OptRand},M) &= \mathrm{pseudoXOF}(\mathtt{tag}_{PRFmsg}\|\mathtt{SK.prf}\|\mathtt{OptRand}\|M,n),\\
\mathbf{F}(\mathtt{PK.seed},\mathtt{ADRS},M_1) &= \mathrm{pseudoXOF}(\mathtt{tag}_{F}\|\mathtt{PK.seed}\|\mathtt{ADRS}\|M_1,n),\\
\mathbf{H}(\mathtt{PK.seed},\mathtt{ADRS},M_1\|M_2) &= \mathrm{pseudoXOF}(\mathtt{tag}_{H}\|\mathtt{PK.seed}\|\mathtt{ADRS}\|M_1\|M_2,n),\\
\mathbf{T}_{\ell}(\mathtt{PK.seed},\mathtt{ADRS},M) &= \mathrm{pseudoXOF}(\mathtt{tag}_{T}\|\mathtt{PK.seed}\|\mathtt{ADRS}\|M,n).
\end{aligned}
$$

其中，`sm3hash` 是固定输出长度的 SM3 接口，而 `pseudohash`/`pseudoXOF` 提供 NGCC API 所需的可变长度哈希与 XOF 行为。当前对外打包实现对这些调用使用显式域分离标签，并保持 address/seed 的输入布局与 ICCS 辅助函数一致。

9 设计依据

Kylin 的设计思路是：沿用原始 SPHINCS$^+$ 的总体结构，引入优化后的 CS-WOTS$^+$ 一次性签名方案，并在超树上采用 Diverse Layer Heights（分层高度多样化）优化。SPHINCS$^+$ 的基本出发点如下：可以通过构造一棵巨大的二叉认证树，并为每个待签消息随机选择一个叶子，来建立无状态的哈希签名方案。然而，这样的树必须极高，其高度通常需要达到大约两倍的安全级别，从而导致签名尺寸极不实用。使用超树代替单棵二叉认证树，可以在速度与签名大小之间进行折中，但仅靠这一点仍不足以同时获得实用的尺寸和速度。

数学困难问题。 Kylin 是一种无状态、抗量子的数字签名方案，其安全性沿用 SPHINCS$^+$ 的证明框架，继承自底层哈希与 XOF 原语在面对通用攻击时的困难性。从构造层面看，Kylin 可被视为对 NIST 标准化 SPHINCS+ 设计的改进：它保留了基于哈希的超树与 FORS 结构，同时将 Winternitz 一次性签名层替换为 CS-WOTS$^+$ 。

参数选择准则。 主参数选择规则是在尽可能接近原始 SPHINCS$^+$ 风格实现与分析方式的前提下，优于可比较的 SPHINCS$^+$ 风格参数配置。在固定的 NGCC 安全级别要求下，搜索优先级依次为：更小的签名、更快的验证、再到更快的签名，同时保持单密钥签名预算为 $q = 2^{64}$。因此，表 3 中选定的实例主要依赖三个设计杠杆：常和 CS-WOTS$^+$ 编码、FORS 参数 (a, k)，以及超树的分层高度配置 $(h_0, \ldots, h_{d-1})$。

由于一次性签名在 SPHINCS$^+$ 超树结构中被大量使用，优化一次性签名的尺寸会直接影响整体性能。借助 CS-WOTS$^+$，Kylin 可以在保持运行时间大致相当的同时，将整个签名方案重新调参为更小的签名。Diverse Layer Heights 进一步消除了 h 必须能被 d 整除这一不必要限制：只要各层高度之和为 h，各层就可以不同，因此搜索过程能够使用像 `Kylin-128` 的 $(7, 7, 7, 7, 7, 7, 7, 8, 8)$ 这类更平衡的配置。

失败概率与安全影响。 Kylin 不存在类似解密或封装场景中的正确性失败事件。对于诚实生成的密钥与诚实生成的签名，验证过程是确定性的；除实现故障或底层密码学假设失效外，验证都会接受。因此，参数搜索中使用的概率项是伪造成功概率上界，而不是正确性失败概率。

对于同一密钥对下签名预算为 q 的场景，参数搜索中使用的具体安全指数由下式给出：

$$b = -\log\left(\frac{1}{2^{8n}} + \sum_{\gamma}\left(1 - \left(1 - \frac{1}{t}\right)^{\gamma}\right)^{k}\binom{q}{\gamma}\left(1 - \frac{1}{2^{h}}\right)^{q-\gamma}\frac{1}{2^{h\gamma}}\right).$$

选定的主实例均取 $q = 2^{64}$，其安全指数如 表 3 所示。相应地，三个主实例的该概率项分别大约被界定为 $2^{-127.209}$、$2^{-255.944}$ 与 $2^{-511.243}$。这些指数用于评估重复 FORS 索引选择及通用伪造事件对所声明安全级别的影响。

其他考虑。 仓库中还保留了 $q = 2^{24}$ 的附加参考参数配置。这些配置不是 NGCC 主提交通道中的实例，但对于单密钥签名预算较小、同时对签名尺寸更敏感的应用场景仍然有参考价值；参见 Appendix B。

我们能够证明，CS-WOTS$^+$ 中使用的编码方案在所有树结构编码方案中都是尺寸最优的，这个结论甚至覆盖了那些不一定能够高效计算的编码方案。进一步地，就我们目前所知，现有任何基于 DAG 的方案都无法在签名尺寸上优于该方案，因此我们的方案可视为已知方案中的尺寸最优者 [30]。我们也注意到基于拒绝采样的概率型编码方案 [18]，并认为这些技术具有独立研究价值。

10 安全性声明与分析

本节作为提交草稿中的安全性声明部分。对于 第7节 中规定的数字签名功能，我们在下述假设与实例化条件下声明其满足 EUF-CMA 安全性。推荐的 NGCC 对齐 Kylin 参数族列于 表 3，打包代码中使用的 ICCS 哈希/XOF 接口说明见 第8.2小节。

理论安全性。 Kylin 延续了 SPHINCS$^+$ 的安全分析框架。因此，该签名方案的 EUF-CMA 安全性应可归约到底层可调哈希函数族的安全性，这些函数族分别用于 $\mathbf{H}_{\mathbf{msg}}$、$\mathbf{PRF}$、$\mathbf{PRF}_{\mathbf{msg}}$、$\mathbf{F}$、$\mathbf{H}$ 和 $\mathbf{T}$。在通常的分析模型中，这些原语被视为理想随机预言机或允许量子访问的随机预言机，相应得到的安全界被解释为针对通用经典攻击与通用量子攻击的安全界。

具体来说，签名方案的 EUF-CMA 安全性依赖于以下底层组件的安全性质：

- $\mathbf{PRF}$ 与 $\mathbf{PRF}_{\mathbf{msg}}$ 的伪随机函数安全性；

- $\mathbf{H}_{\mathbf{msg}}$ 的交错目标子集韧性（ITSR）；
- $\mathbf{F}$ 的单函数多目标不可检测性（SM-UD）、目标碰撞抗性（SM-TCR）以及判定型第二原像抗性（SM-DSPR）；
- $\mathbf{H}$ 与 $\mathbf{T}_\ell$ 的 SM-TCR。

具体安全性与失败概率。 这里的安全分析与 SPHINCS$^+$ 完全一致。Kylin 在面对通用攻击时的经典安全级别（即 bit security）可由下式计算：

$$b = -\log\left(\frac{1}{2^{8n}} + \sum_{\gamma}\left(1-\left(1-\frac{1}{t}\right)^{\gamma}\right)^{k}\binom{q}{\gamma}\left(1-\frac{1}{2^h}\right)^{q-\gamma}\frac{1}{2^{h\gamma}}\right) .$$

Kylin 在面对通用攻击时的量子安全级别可由下式计算：

$$b = -\frac{1}{2}\log\left(\frac{1}{2^{8n}} + \sum_{\gamma}\left(1-\left(1-\frac{1}{t}\right)^{\gamma}\right)^{k}\binom{q}{\gamma}\left(1-\frac{1}{2^h}\right)^{q-\gamma}\frac{1}{2^{h\gamma}}\right) .$$

这里忽略了对数内部的小常数因子。

对于 表 3 中推荐的 NGCC 提交参数族，我们在这些公式中统一取 $q = 2^{64}$，以匹配征集要求中“单个密钥对至少支持 2^{64} 次签名”的明确约束。

其他安全属性。 本草稿目前不额外声明其他形式化安全性质。尤其是，当前第一阶段文本不声明专门的侧信道抗性、完美前向安全性、量化的多密钥归约，或超出已声明单密钥 $q = 2^{64}$ 分析范围之外的临时密钥重复使用安全定理。

ICCS 哈希/XOF 实例化的安全性。 已打包的 NGCC 实现使用 ICCS 辅助函数 `sm3hash`、`pseudohash` 和 `pseudoXOF`。在方案层面，$\mathbf{H}_{\mathbf{msg}}$、$\mathbf{PRF}$、$\mathbf{PRF}_{\mathbf{msg}}$、$\mathbf{F}$、$\mathbf{H}$ 和 $\mathbf{T}$ 都通过基于 `pseudoXOF` 的域分离包装函数来实现，并在输入布局中显式区分标签以及 seed/address。

因此，本草稿中的安全性声明是：在选定的参数规模下，这些具体实例化函数能够提供 Kylin 证明框架所需的伪随机性、目标碰撞抗性及相关性质。除这一标准的通用分析解释之外，本文本并未针对 ICCS 辅助函数内部实现单独提出额外的紧致性声明。

11 性能评估

本节给出征集要求所需的性能分析与基准数据。当前仓库对打包的 Kylin NGCC 实例使用仓库本地测量结果。

性能分析。 在设计层面，Kylin 的主要开销来自哈希与 XOF 调用。密钥生成需要构造超树顶层；签名需要计算 FORS 相关材料，并在超树的每一层上计算一条认证路径；验证则需要重建 FORS 公钥并验证超树签名。CS-WOTS$^+$ 减少了每个一次性签名中 n 字节元素的数量，而 Diverse Layer Heights 则在 h 不能被 d 整除时，仍然使签名阶段的哈希调用代理计数保持平衡。

表 4: 主 Kylin 实例在设计层面的哈希调用与签名尺寸指标。

实例	密钥生成哈希调用	签名哈希调用	验证哈希调用	签名字节数
Kylin-128	417279	2442880	7357	6688
Kylin-256	324095	3223168	12577	26784
Kylin-512	495615	6198272	42476	103808

基准平台。 软件实现使用 ISO C 编写，并采用 GCC 13.3.0。参考实现保持默认的 `-O2` ISO C 编译配置，并启用常规告警检查。优化后的 64 位 PC 实现使用以 `-O3 -march=native` 为核心的目录级构建选项；其中 `Kylin-128` 还额外启用了链接时优化、省略帧指针、循环展开以及 `-fno-plt`。基准主机为 x86-64 64 位 PC 环境：AMD Ryzen 7 7800X3D 8-Core Processor，8 个物理核心、16 个硬件线程，`/proc/cpuinfo` 报告主频约为 4.192 GHz，内存 30 GiB，操作系统为 Ubuntu 24.04.3 LTS，运行于 Linux 6.6.87.2-microsoft-standard-WSL2。实现中使用 ICCS 辅助哈希/XOF 接口 `sm3hash`、`pseudohash` 与 `pseudoXOF`；报告中的打包基准不依赖外部密码学库。

软件性能、传输与存储开销。 表 5 中的自动化打包测量结果由当前实现树上的 package benchmark 驱动脚本生成。该表给出在 32 字节消息上、经过 3 次预热后的 9 次实测主机本地墙钟时间中位数（毫秒），并同时列出公钥、私钥和签名大小（字节）。由于这些数据是仓库本地环境中的墙钟测量值，其绝对数值应理解为当前主机上的实测点，而不是跨平台的统一性能声明。

在本次重新测试中，优化实现相较参考实现仍保持一致优势。对于 `Kylin-128`、`Kylin-256` 和 `Kylin-512`，签名阶段的实测加速比分别约为 3.06×、2.84× 和 3.16×；密钥生成阶段分别约为 3.85×、4.28× 和 5.89×；验证阶段分别约为 2.58×、2.75× 和 3.77×。

表 5: 已打包 Kylin NGCC 实例在仓库本地环境中的软件性能与传输/存储开销。时间单位为 32 字节消息上的毫秒；尺寸单位为字节。

实例	实现	密钥生成 (ms)	签名 (ms)	验证 (ms)	公钥	私钥	签名
Kylin-128	opt	75.058	592.035	1.989	32	64	6688
Kylin-128	ref	288.953	1813.298	5.141	32	64	6688
Kylin-256	opt	52.720	878.207	3.299	64	128	26784
Kylin-256	ref	225.417	2491.528	9.088	64	128	26784
Kylin-512	opt	171.051	4369.834	22.940	128	256	103808
Kylin-512	ref	1006.842	13801.995	86.494	128	256	103808
Kylin-128-q24	opt	344659	546652	0.147	32	64	3840
Kylin-128-q24	ref	853437	1072184	0.201	32	64	3840
Kylin-256-q24	opt	348092	1259275	0.385	64	128	14880
Kylin-256-q24	ref	859058	1875189	0.522	64	128	14880
Kylin-512-q24	opt	1092497	3311972	5.524	128	256	52672
Kylin-512-q24	ref	2731243	6623944	8.286	128	256	52672

12 特性说明

本节按照征集要求，对 Kylin 的主要特性进行归纳说明。

创新性。 Kylin 引入了 CS-WOTS$^+$，这是一种适用于 Winternitz 类一次性签名的常和、尺寸最优编码方式，并将其与超树参数搜索中的 Diverse Layer Heights 结合使用。这两项改动在保持保守的哈希签名设计路线的同时，改进了签名尺寸与哈希调用之间的折中关系。

简洁性。 该方案仍然完全处于研究充分的无状态哈希签名家族之内。其核心组成包括可调哈希函数、一次性签名层、FORS 与超树，不涉及格、码或多变量代数结构。

灵活性。 参数 (n, w, h, d, k, t) 允许构造在签名尺寸、签名时间与验证时间之间进行折中。本次主提交固定为 `Kylin-128`、`Kylin-256` 与 `Kylin-512` 三个配置，而附录中记录了仅供参考的 $q = 2^{24}$ 配置，用于更低签名预算场景。

兼容性。 已打包代码遵循 NGCC SIG API，并通过 ICCS 辅助函数路由哈希/XOF 调用。对外工作流保持为常规的密钥生成、签名与验证接口。由于 Kylin 用 CS-WOTS$^+$ 替换了 WOTS$^+$，因此它与原始 SPHINCS$^+$ 的字节级格式并不完全兼容。

可扩展性。 该构造具有模块化特征：替换哈希/XOF 实例化或重新调整参数集，都不会改变顶层签名工作流。不过，当前提交仅声明数字签名这一种功能。

实现性能优势。 仓库同时提供三组主实例的 ISO C 参考实现与优化后的 64 位 PC 实现。由 表 5 可见，在当前 64 位 PC 基准平台上，优化实现相较参考实现提升了密钥生成、签名与验证的实测性能。

参考文献

[1] Désiré André. Mémoire sur les combinaisons régulières et leurs applications. In *Annales scientifiques de l'École Normale Supérieure*, volume 5, pages 155–198, 1876.

[2] Jean-Philippe Aumasson and Guillaume Endignoux. Improving stateless hash-based signatures. In Nigel P. Smart, editor, *Topics in Cryptology – CT-RSA 2018*, volume 10808 of *Lecture Notes in Computer Science*, pages 219–242, San Francisco, CA, USA, April 16–20, 2018. Springer, Cham, Switzerland.

[3] Hacene Belbachir and Oussama Igueroufa. Congruence properties for bi s nomial coefficients and like extended ram and kummer theorems under suitable hypothesis. *Mediterranean Journal of Mathematics*, 17(1):1–14, 2020.

[4] Daniel J. Bernstein, Daira Hopwood, Andreas Hülsing, Tanja Lange, Ruben Niederhagen, Louiza Papachristodoulou, Michael Schneider, Peter Schwabe, and Zooko Wilcox-O'Hearn. SPHINCS: Practical stateless hash-based signatures. In Elisabeth Oswald and Marc Fischlin, editors, *Advances in Cryptology – EUROCRYPT 2015, Part I*, volume 9056 of *Lecture Notes in Computer Science*, pages 368–397, Sofia, Bulgaria, April 26–30, 2015. Springer Berlin Heidelberg, Germany.

[5] Daniel J. Bernstein, Andreas Hülsing, Stefan Kölbl, Ruben Niederhagen, Joost Rijneveld, and Peter Schwabe. The SPHINCS$^+$ signature framework. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019: 26th Conference on Computer and Communications Security*, pages 2129–2146, London, UK, November 11–15, 2019. ACM Press.

[6] Daniel Bleichenbacher and Ueli M. Maurer. Directed acyclic graphs, one-way functions and digital signatures. In Yvo Desmedt, editor, *Advances in Cryptology – CRYPTO'94*, volume 839 of *Lecture Notes in Computer Science*, pages 75–82, Santa Barbara, CA, USA, August 21–25, 1994. Springer Berlin Heidelberg, Germany.

[7] Jurjen N. Bos and David Chaum. Provably unforgeable signatures. In Ernest F. Brickell, editor, *Advances in Cryptology – CRYPTO'92*, volume 740 of *Lecture Notes in Computer Science*, pages 1–14, Santa Barbara, CA, USA, August 16–20, 1993. Springer Berlin Heidelberg, Germany.

[8] David A Cooper, Daniel C Apon, Quynh H Dang, Michael S Davidson, Morris J Dworkin, Carl A Miller, et al. Recommendation for stateful hash-based signature schemes. *NIST Special Publication*, 800:208, 2020.

[9] Robert P Dilworth. A decomposition theorem for partially ordered sets. In *Classic Papers in Combinatorics*, pages 139–144. Springer, 2009.

[10] Leonhard Euler. De evolutione potestatis polynomialis cuiuscunque (1+ x+ x 2+ x 3+ x 4+ etc.) n. *Nova Acta Academiae Scientiarum Imperialis Petropolitanae*, pages 47–57, 1801.

[11] Oded Goldreich. Two remarks concerning the Goldwasser-Micali-Rivest signature scheme. In Andrew M. Odlyzko, editor, *Advances in Cryptology – CRYPTO'86*, volume 263 of *Lecture Notes in Computer Science*, pages 104–110, Santa Barbara, CA, USA, August 1987. Springer Berlin Heidelberg, Germany.

[12] Oded Goldreich. *Foundations of Cryptography: Basic Applications*, volume 2. Cambridge University Press, Cambridge, UK, 2004.

[13] Shay Gueron and Nicky Mouha. SPHINCS-simpira: Fast stateless hash-based signatures with post-quantum security. Cryptology ePrint Archive, Report 2017/645, 2017.

[14] Douglas Zare (https://math.stackexchange.com/users/8345/douglas zare). How to express $(1 + x + x^2 + \cdots + x^m)^n$ as a power series? Mathematics Stack Exchange, 2011. URL:https://math.stackexchange.com/q/28861 (version: 2011-11-15).

[15] Andreas Hülsing, Denis Butin, Stefan-Lukas Gazdag, Joost Rijneveld, and Aziz Mohaisen. Xmss: extended merkle signature scheme. In *RFC 8391*. IRTF, 2018.

[16] Andreas Hülsing, Mikhail A. Kudinov, Eyal Ronen, and Eylon Yogev. SPHINCS+C: Compressing SPHINCS+ with (almost) no cost. In *2023 IEEE Symposium on Security and Privacy*, pages 1435–1453, San Francisco, CA, USA, May 21–25, 2023. IEEE Computer Society Press.

[17] Dmitry Khovratovich, Mikhail A. Kudinov, and Benedikt Wagner. At the top of the hypercube - better size-time tradeoffs for hash-based signatures. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology – CRYPTO 2025, Part VI*, volume 16005 of *Lecture Notes in Computer Science*, pages 93–123, Santa Barbara, CA, USA, August 17–21, 2025. Springer, Cham, Switzerland.

[18] Mikhail Kudinov, Andreas Hülsing, Eyal Ronen, and Eylon Yogev. Sphincs+c: Compressing sphincs+ with (almost) no cost. Cryptology ePrint Archive, Paper 2022/778, 2022. `https://eprint.iacr.org/2022/778`.

[19] Leslie Lamport. Constructing digital signatures from a one-way function. Technical Report SRI-CSL-98, SRI International Computer Science Laboratory, October 1979.

[20] David McGrew, Michael Curcio, and Scott Fluhrer. Leighton-micali hash-based signatures. In *RFC 8554*. IRTF, 2019.

[21] Ralph C. Merkle. A certified digital signature. In Gilles Brassard, editor, *Advances in Cryptology – CRYPTO'89*, volume 435 of *Lecture Notes in Computer Science*, pages 218–238, Santa Barbara, CA, USA, August 20–24, 1990. Springer, New York, USA.

[22] National Institute of Standards and Technology. Module-lattice-based digital signature standard (fips 205). Technical report, U.S. Department of Commerce, 2024.

[23] National Institute of Standards and Technology. Recommendation for stateful hash-based signature schemes (initial public draft). Technical report, U.S. Department of Commerce, April 2026. Published April 13, 2026.

[24] Lucas Pandolfo Perin, Gustavo Zambonin, Ricardo Custódio, Lucia Moura, and Daniel Panario. Improved constant-sum encodings for hash-based signatures. *Journal of Cryptographic Engineering*, pages 1–23, 2021.

[25] Zhen Qin and Siwei Sun. Extending the SPHINCS+ framework: Varying the tree heights and chain lengths. Cryptology ePrint Archive, Report 2025/2236, 2025.

[26] Leonid Reyzin and Natan Reyzin. Better than biba: Short one-time signatures with fast signing and verifying. Cryptology ePrint Archive, Report 2002/014, 2002.

[27] Emanuel Sperner. Ein satz über untermengen einer endlichen menge. *Mathematische Zeitschrift*, 27(1):544–548, 1928.

[28] Serge Vaudenay. One-time identification with low memory. In *Eurocode'92*, pages 217–228. Springer, 1993.

[29] S Ole Warnaar. The andrews–gordon identities and q-multinomial coefficients. *Communications in mathematical physics*, 184(1):203–232, 1997.

[30] Kaiyi Zhang, Hongrui Cui, and Yu Yu. Revisiting the constant-sum winternitz one-time signature with applications to sphincs+ and xmss. In *Advances in Cryptology–CRYPTO 2023: 43th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19–24, 2023, Proceedings, to appear*, pages xxx–xxx. Springer, 2023.

A 参数评估 Sage 脚本

下面的脚本用于评估满足 $\sum_{i=0}^{d-1} h_i = h$ 的分层超树高度参数候选 $(h_0, \ldots, h_{d-1})$。其中，消息摘要索引拆分使用 h_0，HT 哈希调用代理计数使用 $\sum_{i=0}^{d-1} 2^{h_i}$。当前冻结行既包括已打包的 $q = 2^{64}$ 参数组 `Kylin-128/256/512`，也包括 $q = 2^{24}$ 的附加参考参数组 `Kylin-128-q24/256-q24/512-q24`。

```
#!/usr/bin/env python3
"""Reference parameter-evaluation script with per-layer HT heights h_vec.

This appendix script mirrors the frozen NGCC parameter set used by the current
repository package:
  - q = 2^64 signatures per key pair
  - each family has its own fixed tuple (h, d, h_vec, a, k, w, len)
  - digest split uses h_0 (lowest layer)
  - HT signing proxy uses sum_i 2^{h_i}
"""

from __future__ import annotations

from functools import lru_cache
```

```
import mpmath as mp

mp.mp.dps = 80

Q_SIGS = 2**64
MAX_LAYER_HEIGHT = 32

FAMILIES = (
    {
        "name": "Kylin-128",
        "n": 16,
        "target_classical": 128,
        "ngcc_quantum": 80,
        "h": 65,
        "d": 9,
        "h_vec": (7, 7, 7, 7, 7, 7, 7, 8, 8),
        "a": 13,
        "k": 11,
        "w": 73,
        "len": 22,
    },
    {
        "name": "Kylin-256",
        "n": 32,
        "target_classical": 256,
        "ngcc_quantum": 128,
        "h": 68,
        "d": 10,
        "h_vec": (6, 6, 7, 7, 7, 7, 7, 7, 7, 7),
        "a": 13,
        "k": 22,
        "w": 54,
        "len": 46,
    },
    {
        "name": "Kylin-512",
        "n": 64,
        "target_classical": 512,
        "ngcc_quantum": 256,
        "h": 70,
        "d": 4,
        "h_vec": (17, 17, 18, 18),
        "a": 20,
        "k": 27,
        "w": 64,
        "len": 87,
    },
)


def balanced_profile(h: int, d: int) -> tuple[int, ...]:
    base = h // d
    rem = h % d
    return tuple(base + (1 if i < rem else 0) for i in range(d))


def shift_height(profile: tuple[int, ...], src: int, dst: int, amount: int) -> tuple[int, ...] | None:
    if amount <= 0:
        return profile
    if src == dst:
        return profile
    p = list(profile)
    if p[src] - amount < 1:
        return None
    p[src] -= amount
    p[dst] += amount
    return tuple(p)


def profile_ok(h: int, h_vec: tuple[int, ...]) -> bool:
    if not h_vec:
        return False
    if any(v < 1 for v in h_vec):
        return False
    if sum(h_vec) != h:
        return False
    if max(h_vec) > MAX_LAYER_HEIGHT:
        return False
    # tree_idx fits in 64 bits for the current implementation split.
    if h - h_vec[0] > 64:
```

```
        return False
    return True


def generate_profiles(h: int, d: int) -> list[tuple[int, ...]]:
    core = balanced_profile(h, d)
    profiles: set[tuple[int, ...]] = {core}

    top = d - 1
    mid = d // 2
    for candidate in (
        shift_height(core, top, 0, 1),
        shift_height(core, top, 0, 2),
        shift_height(core, 0, top, 1),
        shift_height(core, 0, top, 2),
        shift_height(core, mid, 0, 1),
        shift_height(core, mid, top, 1),
        shift_height(core, top, mid, 1),
        shift_height(core, 0, mid, 1),
    ):
        if candidate is not None and profile_ok(h, candidate):
            profiles.add(candidate)

    return sorted(profiles)


@lru_cache(maxsize=None)
def const_sum_count(length: int, w: int) -> int:
    total = length * (w - 1) // 2
    dp = [0] * (total + 1)
    dp[0] = 1
    for _ in range(length):
        nxt = [0] * (total + 1)
        window = 0
        for idx in range(total + 1):
            window += dp[idx]
            if idx - w >= 0:
                window -= dp[idx - w]
            nxt[idx] = window
        dp = nxt
    return dp[total]


@lru_cache(maxsize=None)
def wotscs_length(message_bits: int, w: int) -> int:
    for length in range(2, 700):
        if mp.log(const_sum_count(length, w), 2) > message_bits:
            return length
    raise RuntimeError(f"failed to find WOTSCS length for {message_bits=} {w=}")


@lru_cache(maxsize=None)
def qhit_distribution(h: int, limit: int = 220) -> tuple[mp.mpf, ...]:
    p = mp.mpf(2) ** (-h)
    q = mp.mpf(Q_SIGS)
    cur = mp.e ** (q * mp.log1p(-p))
    probs = [cur]
    for r in range(limit):
        cur *= ((q - r) / (r + 1)) * (p / (1 - p))
        probs.append(cur)
    return tuple(probs)


@lru_cache(maxsize=None)
def classical_security(n: int, h: int, a: int, k: int) -> mp.mpf:
    total = mp.mpf(2) ** (-8 * n)
    tree_size = mp.mpf(2) ** a
    for r, qprob in enumerate(qhit_distribution(h)):
        total += qprob * (1 - (1 - 1 / tree_size) ** r) ** k
    return -mp.log(total, 2)


def minimal_k(target_bits: int, n: int, h: int, a: int) -> int:
    upper = 240 if target_bits < 512 else 420
    lower = 10 if target_bits < 512 else 30
    if classical_security(n, h, a, upper) < target_bits:
        raise RuntimeError(f"no feasible k found for n={n}, h={h}, a={a}")

    lo, hi = lower, upper
    while lo < hi:
        mid = (lo + hi) // 2
```

```
        if classical_security(n, h, a, mid) >= target_bits:
            hi = mid
        else:
            lo = mid + 1
    return lo


def signature_bytes(n: int, h: int, d: int, a: int, k: int, length: int) -> int:
    return ((a + 1) * k + h + length * d + 1) * n


def hash_calls(h_vec: tuple[int, ...], a: int, k: int, w: int, length: int) -> int:
    return k * (2 ** (a + 1)) + sum((2**layer_h) * (length * w + 1) for layer_h in h_vec)


def digest_bytes(k: int, a: int, h: int, h0: int) -> int:
    tree_bits = h - h0
    return ((k * a + 7) // 8) + ((tree_bits + 7) // 8) + ((h0 + 7) // 8)


def choose_profile(h: int, d: int, a: int, k: int, w: int, length: int) -> tuple[int, ...]:
    profiles = generate_profiles(h, d)
    return min(profiles, key=lambda vec: (hash_calls(vec, a, k, w, length), vec))


def evaluate_family(family: dict[str, int | str | tuple[int, ...]]) -> dict[str, int | float | str]:
    h = int(family["h"])
    d = int(family["d"])
    h_vec = tuple(int(x) for x in family["h_vec"])  # type: ignore[arg-type]
    n = int(family["n"])
    a = int(family["a"])
    k = int(family["k"])
    w = int(family["w"])
    length = int(family["len"])

    return {
        "name": str(family["name"]),
        "n": n,
        "h": h,
        "d": d,
        "h_vec": h_vec,
        "h0": h_vec[0],
        "a": a,
        "k": k,
        "w": w,
        "len": length,
        "classical": float(classical_security(n, h, a, k)),
        "quantum_formula": float(classical_security(n, h, a, k) / 2),
        "ngcc_quantum": int(family["ngcc_quantum"]),
        "pk": 2 * n,
        "sk": 4 * n,
        "sig": signature_bytes(n, h, d, a, k, length),
        "digest_bytes": digest_bytes(k, a, h, h_vec[0]),
        "hashcalls": hash_calls(h_vec, a, k, w, length),
    }


def verify_frozen_parameters(rows: list[dict[str, int | float | str]]) -> None:
    expected: dict[str, tuple[int, int, tuple[int, ...], int, int, int, int]] = {
        "Kylin-128": (65, 9, (7, 7, 7, 7, 7, 7, 7, 8, 8), 13, 11, 73, 22),
        "Kylin-256": (68, 10, (6, 6, 7, 7, 7, 7, 7, 7, 7, 7), 13, 22, 54, 46),
        "Kylin-512": (66, 11, (6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6), 13, 49, 97, 79),
    }
    for row in rows:
        name = str(row["name"])
        if name not in expected:
            raise AssertionError(f"unexpected family: {name}")

        exp_h, exp_d, exp_h_vec, exp_a, exp_k, exp_w, exp_len = expected[name]
        h = int(row["h"])
        d = int(row["d"])
        length = int(row["len"])
        a = int(row["a"])
        k = int(row["k"])
        w = int(row["w"])
        h_vec = tuple(row["h_vec"])  # type: ignore[arg-type]

        if (h, d, h_vec, a, k, w, length) != (exp_h, exp_d, exp_h_vec, exp_a, exp_k, exp_w, exp_len):
            raise AssertionError(f"parameter drift for {name}: {(h, d, h_vec, a, k, w, length)}")

        recomputed_hashcalls = hash_calls(h_vec, a, k, w, length)
```

```
        if int(row["hashcalls"]) != recomputed_hashcalls:
            raise AssertionError(f"hashcalls mismatch for {row['name']}")


def main() -> None:
    rows = [evaluate_family(family) for family in FAMILIES]
    verify_frozen_parameters(rows)

    for row in rows:
        print(
            f"{row['name']}: h={row['h']}, d={row['d']}, h_vec={row['h_vec']}, "
            f"a={row['a']}, k={row['k']}, w={row['w']}, len={row['len']}, "
            f"classical={row['classical']:.1f}, ngcc_quantum={row['ngcc_quantum']}, "
            f"formula_quantum={row['quantum_formula']:.1f}, "
            f"pk={row['pk']}, sk={row['sk']}, sig={row['sig']}, "
            f"digest_bytes={row['digest_bytes']}, hashcalls={row['hashcalls']}"
        )


if __name__ == "__main__":
    main()
```

B $q = 2^{24}$ 附加参数配置

本附录汇总了冻结的 $q = 2^{24}$ 附加参数配置，它们在仓库中作为额外的参考实现保留。其动机是实践性的：在某些应用场景中，当单密钥签名预算极大时，现有签名尺寸与验证时间行为仍难以令人满意。此外，NIST 于 2026 年 4 月 13 日发布的 SP 800-230 初始公开草案强调了带有显式单密钥使用上界的有状态哈希签名，这也进一步说明了为受限签名次数工作点（包括 2^{24} 这一类限制）保留明确附加参数行的意义 [23]。

在本仓库中，这些 $q = 2^{24}$ 配置通过与主线相同的受约束搜索工作流选出，随后被冻结为已打包 NGCC $q = 2^{64}$ 参数族的参考性附加配置。

表 7 中的基准行同步自 `package_bench.csv` 中的 $q = 2^{24}$ 条目。

优化实现采用宏展开的 SM3 压缩函数（64 个显式轮函数，无分支）、面向消息字加载的 SSE2/AVX2 硬件分派、针对字节对齐输入的无 `malloc` `pseudoXOF` 快速路径，以及适配 $n = 16/32/64$ 的 4 路 SSE2 WOTS 链加速（`thash_wots_chain_4x`）。签名与验证路径均以每次 SIMD 操作处理 4 条 WOTS 链。每个优化变体的 KAT 输出均与对应参考实现逐字节一致（经 SHA-256 校验）。

算法 21: kylin_sign — 生成 Kylin 签名。

```
# Input: Message M, private key SK = (SK.seed, SK.prf, PK.seed, PK.root)
# Output: Kylin signature SIG
kylin_sign(M, SK){ // init
ADRS = toByte(0, 32);
  // generate randomizer
  opt = PK.seed;
  if(RANDOMIZE){
    opt = rand(n);
  }
  R = PRF_msg(SK.prf, opt, M);
  SIG = SIG  R;
  // compute message digest and index
  digest = H_msg(R, PK.seed, PK.root, M);
  tmp_md = first floor((ka +7)/ 8) bytes of digest;
  tmp_idx_tree = next floor((h - h_0 +7)/ 8) bytes of digest;
  tmp_idx_leaf = next floor((h_0 +7)/ 8) bytes of digest;
  md = first ka bits of tmp_md;
  idx_tree = first h - h_0 bits of tmp_idx_tree;
  idx_leaf = first h_0 bits of tmp_idx_leaf;
  // FORS sign
  ADRS.setLayerAddress(0);
  ADRS.setTreeAddress(idx_tree);
  ADRS.setType(FORS_TREE);
  ADRS.setKeyPairAddress(idx_leaf);
  SIG_FORS = fors_sign(md, SK.seed, PK.seed, ADRS);
  SIG = SIG  SIG_FORS;
  // get FORS public key
  PK_FORS = fors_pkFromSig(SIG_FORS, md, PK.seed, ADRS);
  // sign FORS public key with HT
  ADRS.setType(TREE);
  SIG_HT = ht_sign(PK_FORS, SK.seed, PK.seed, idx_tree, idx_leaf);
  SIG = SIG  SIG_HT;
  return SIG;
}
```

算法 22: `kylin_verify` — 使用 Kylin 公钥 **PK** 验证消息 M 上的 Kylin 签名 **SIG**。

```
# Input: Message M, signature SIG, public key PK
# Output: Boolean
kylin_verify(M, SIG, PK){ // init
  ADRS = toByte(0, 32);
  R = SIG.getR();
  SIG_FORS = SIG.getSIG_FORS();
  SIG_HT = SIG.getSIG_HT();
  // compute message digest and index
  digest = H_msg(R, PK.seed, PK.root, M);
  tmp_md = first floor((ka +7)/ 8) bytes of digest;
  tmp_idx_tree = next floor((h - h_0 +7)/ 8) bytes of digest;
  tmp_idx_leaf = next floor((h_0 +7)/ 8) bytes of digest;
  md = first ka bits of tmp_md;
  idx_tree = first h - h_0 bits of tmp_idx_tree;
  idx_leaf = first h_0 bits of tmp_idx_leaf;
  // compute FORS public key
  ADRS.setLayerAddress(0);
  ADRS.setTreeAddress(idx_tree);
  ADRS.setType(FORS_TREE);
  ADRS.setKeyPairAddress(idx_leaf);
  PK_FORS = fors_pkFromSig(SIG_FORS, md, PK.seed, ADRS);
  // verify HT signature
  ADRS.setType(TREE);
  return ht_verify(PK_FORS, SIG_HT, PK.seed, idx_tree, idx_leaf, PK.root);
}
```

表 6: 冻结的 $q = 2^{24}$ Kylin 附加参数配置（仅供参考的额外实现）。

参数族	NGCC 等级	n	h	d	$\log t$	k	w	l	bitsec	pk	sk	sig
Kylin-128-q24	128/80	16	22	1	24	6	4	67	127	32	64	3840
Kylin-256-q24	256/128	32	21	1	25	12	4	131	255	64	128	14880
Kylin-512-q24	512/256	64	57	3	20	24	64	87	511	128	256	52672

表 7: $q = 2^{24}$ Kylin 附加实现的打包基准结果。时间单位为 32 字节消息上的毫秒；签名尺寸单位为字节。

参数族	实现	密钥生成 (ms)	签名 (ms)	验证 (ms)	签名 (B)
Kylin-128-q24	ref	853,437	1,072,184	0.201	3,840
Kylin-128-q24	opt	344,659	546,652	0.147	3,840
Kylin-256-q24	ref	859,058	1,875,189	0.522	14,880
Kylin-256-q24	opt	348,092	1,259,275	0.385	14,880
Kylin-512-q24	ref	2,731,243	6,623,944	8.286	52,672
Kylin-512-q24	opt	1,092,497	3,311,972	5.524	52,672